SZENT ISTVÁN KATOLIKUS
TECHNIKUM ÉS GIMNÁZIUM

CLANVERSE

SZOFTVERFEJLESZTÉS ÉS -TESZTELÉS

VIZSGAREMEK

Készítették:

BÓDISZ LORÁND

SEDLÁK ÁKOS

SIMKÓ ROLAND

Sátoraljaújhely, 2026

Tartalomjegyzék

I. Bevezetés ... 6
II. Főbb Funkciók Részletesen: ... 7
III. Fejlesztéshez használt eszközök és technológiák ... 8
3.1 Fejlesztői Eszköztár: ... 8
3.2 Csoportmunkát segítő szoftverek: ... 8
3.3 Dokumentációkészítéshez használt eszközök: ... 8
3.4 Képtervezéshez és szerkesztéshez használt szoftverek ... 8
3.5 Backend Technológiák és Keretrendszerek: ... 10
3.6 Frontend Technológiák: ... 10
IV. Felhasználói dokumentáció ... 12
4.1. Rendszerkövetelmények: ... 12
Minimális ... 12
Ajánlott ... 12
4.2. Előfeltételek: ... 13
4.3. Projekt klónozása ... 13
4.3.1. Letöltés Github oldalról ... 13
4.3.2. Letöltés Github Desktop alkalmazással ... 17
4.4. Projekt megnyitása Visual Studio Code-ban ... 20
4.5. Backend Beállítása ... 21
4.5.1 Parancssor megnyitása ... 21
4.5.2 Környezeti változók beállítása ... 22
4.5.3 Adatbázis beállítása ... 23
4.5.4 Backend szerver elindítása ... 26
4.6 Frontend beállítása ... 26
4.6.1 Parancssor megnyitása ... 26
4.6.2 Angular és csomagok telepítése ... 27
4.6.3 Frontend elindítása ... 27
4.7 Minden porton futó projekt beállítása ... 28
4.7.1 Localhost leállítása: ... 28
4.7.2 Minden porton figyelő szerver indítása: ... 29
4.7.3 Frontend konfigurálása és indítása: ... 29

V. Backend architektúra és üzleti logika 33
VI. Adatbázis dokumentáció 34
6.1. Users (Felhasználók) Tábla: 34
6.2. Games (Játékok) Tábla: 34
6.3. Clans (Klánok) Tábla 34
6.4. ClanMembers (Klántagok) Tábla 35
6.5. ClanMessages (Üzenetek) Tábla 35
VII. Biztonsági megoldások és adatvédelem 36
VIII. Részletes backend API dokumentáció 37
8.1. Felhasználó Kezelés (/users) 37
8.2. Klánok Kezelése (/clans) 38
8.3. Üzenetek és Chat (/messages) 40
8.4. Játékok (/games) 40
8.5 Index (/) 41
8.6 Admin (/admin) 41
IX. Frontend Dokumentáció 42
9.1 Bevezetés 42
9.2 Útvonalazás és navigáció 42
9.2.1 Az alkalmazás útvonalválasztása 42
9.2.2. Guard-ok és autentikációs védelem 43
9.3 Hitelesítési rendszer 43
9.3.1 JWT –alapú hitelesítés: 43
9.3.2 HTTP Interceptor az automatikus token kezeléshez: 43
9.3.3 Felhasználói hitelesítési folyamat: 43
9.4 Alkalmazás-wide szolgáltatások (services) 43
9.4.1 UserService – Felhasználó kezelési szolgáltatás: 43
9.4.2 ClanService – Klán kezelési szolgáltatás: 44
9.4.3 MessageService – Üzenetkezelési szolgáltatás: 45
9.4.4 GameService – Játék kezelési szolgáltatás: 45
9.4.5 AdminService – Adminisztratív funkciók kezelése: 46
9.5 Felhasználófelületi komponensek részletesen 46
9.5.1 App komponens – Az alkalmazás gyökérkomponense: 46
9.5.2 Főoldal komponens – A kezdőoldal: 46

9.5.3 Bejelentkezés komponens – Autentikációs felület: 47
9.5.4 Regisztráció komponens – Regisztrációs felület: 48
9.5.5 Klánok komponens – Klán böngészési és kezelési oldal: 49
9.5.6 Klán létrehozás komponens- Új klán létrehozás: 51
9.5.7 Profil komponens – Felhasználói profil szerkesztés: 52
9.5.8 Message komponens – Klánspecifikus üzenetcsatorna: 54
9.5.9 Admin komponens - Adminisztratív panel: 55
9.5.10 Ászf komponens – Általános Szerződési Feltételek: 56
9.5.11 Navbar komponens – Navigációs sáv: 56
9.5.12 Footer komponens – Alkalmazás lábléce: 57
9.6 Alkalmazás konfigurációi és inicializálás 58
9.6.1 App config – Az alkalmazás konfigurációs fájlja: 58
9.7 Styling és CSS keretrendszer 58
9.8 HTTP kommunikáció és API integrációs minta 58
9.8.1 API végpontok és útvonalak: 58
9.8.2 Error handling és hibakezelés: 59
9.9 Forma kezelés és validáció 59
9.10 Biztonsági szempontok és lehetséges sebezhetőségek 59
9.10.1 Token tárolás biztonsága: 59
9.10.2 Jelszókezelés: 60
9.10.3 Content Security Policy (CSP): 60
9.11 Teljesítmény optimalizálási módszerek 60
9.11.1 Lazy Loading komponensek: 60
9.12 Összefoglaló 60
X. Projekt szerkezete 61
XI. Munkamenet és csapatfelosztás 62
XII. Felhasználói esetek és követelmények (User Stories) 63
XIII. Szoftvertesztelés jegyzőkönyv 63
XIV. Kódanalízis (backend és adatbázis) 64
14.1. Adatbázis-kapcsolat és szinkronizáció 64
14.2. Dinamikus Adatmodellek és Táblakapcsolatok 65
14.3. Hitelesítés, Jogosultság és Végpontvédelem (Auth Middleware) 66
14.4. Szigorúan Ellenőrzött Fájlfeltöltés (Multer) 67

14.5. Komplex Üzleti Logika: Fail-Fast (Gyors Bukás) 68
14.6. A Központosított Hibakezelés Elve 69
14.7. Fő Szerver Konfiguráció és Biztonsági Házirend (CORS) 70
14.8. Adatbázis Inicializálás és Biztonságos Seeding 71
XV. Továbbfejlesztési lehetőségek 72
XVI. Források 73
16.1 Bódisz Loránd források 73
16.2 Sedlák Ákos források 73
16.3 Simkó Roland források 74

I. Bevezetés

A vizsgaremeken dolgozó csapatunk mindhárom tagjának életében meghatározó szerepet tölt be a videójátékozás, mint hobbi és közös érdeklődési kör. Legyen szó kooperatív (multiplayer) vagy egyjátékos (singleplayer) élményekről, FPS, MOBA, vagy éppen más műfajú alkotásokról, a gamer közeg mindannyiunkhoz nagyon közel áll. Ebből a személyes indíttatásból fakadóan döntöttünk úgy, hogy vizsgamunkaként egy olyan hiánypótló webalkalmazást készítünk el, amely erre a szubkultúrára épül és valós játékosi igényeket szolgál ki.

A projekt elnevezésekor egy egyszerű, könnyen megjegyezhető, ugyanakkor lényegre törő nevet kerestünk. A „Clan” szó utal arra, hogy csapatokról, csapatszervezésről és közösségről van szó, a „Verse” pedig az Univerzum (universe) szóból ered, így a projekt magyarosított elnevezése a „Klánok univerzuma”.

A ClanVerse egy modern, webalapú közösségi platform, amely kifejezetten a gamer társadalom számára készült. A projekt elsődleges célja, hogy egy gyors és biztonságos felületet biztosítson az e-sportolók, hobbi játékosok és baráti társaságok számára, ahol könnyedén alapíthatnak klánokat, menedzselhetik csapataikat, és tarthatják a kapcsolatot a tagokkal.

Napjainkban a játékosok számtalan platformot (Discord, TeamSpeak, játékokon belüli felületek) használnak a kommunikációra, azonban a ClanVerse egy dedikált, átfogó webes környezetet teremt, ahol a klánok adminisztrációja és a csapattagok kezelése egyetlen központosított helyen történik. A webalkalmazás reszponzív kialakításának köszönhetően asztali számítógépen és mobiltelefonon egyaránt teljes értékű felhasználói élményt nyújt.

II. Főbb Funkciók Részletesen:

- Felhasználói fiókok kezelése: Komplex regisztrációs és bejelentkezési rendszer, biztonságos jelszókezeléssel (Bcrypt titkosítás) és JWT (JSON Web Token) alapú, állapotmentes (stateless) munkamenet-kezeléssel. Profilok testreszabása profilképek biztonságos, validált szerveroldali feltöltésével.
- Klánok teljes körű menedzselése: A felhasználók saját klánokat hozhatnak létre, meghatározhatják a klán nevét, leírását, és hozzárendelhetik egy adott játékhoz. A klán alapítója automatikusan "Leader" (vezető) jogosultságot kap, amellyel teljes kontrollt gyakorol a csapat felett.
- Klántársaság és Jogosultságok: Dinamikus csatlakozási rendszer. A tagok szabadon kiléphetnek, a vezetők pedig kizárhatnak tagokat. A rendszer szigorúan ellenőrzi a műveletek végrehajtásához szükséges hierarchikus jogosultságokat az adatbázis relációin keresztül.
- Valós idejű és aszinkron Chat: A klántagok egy dedikált üzenőfalon tarthatják a kapcsolatot. Az üzeneteket a szerver adatbázisban rögzíti, így a kommunikáció maradandó, a korábbi üzenetváltások bármikor visszakereshetők és szinkronizáltak az összes kliens között.
- Játékok adatbázisa: Központi játékkatalógus, amelyből a klánok választhatnak. A játékok adataihoz (név, logó) a rendszer optimalizált fájlkezelést és relációs adatintegritást biztosít.
- Adminisztrációs Panel: Egy védett, csak "Admin" jogosultsággal rendelkező felhasználók számára elérhető felület, ahol az oldal globális adatai (például új játékok feltöltése a központi katalógusba) menedzselhetők. Minden egyéb adminisztrációs feladat a héttérben végezhető el (kódban)

III. Fejlesztéshez használt eszközök és technológiák

A projektünk elkészítéséhez a vizsgakövetelmény által megkövetelt modern technológiákat és egyéb eszközöket használtunk.

3.1 Fejlesztői Eszköztár:

- Visual Studio Code: Pehelysúlyú, mégis rendkívül bővíthető kódszerkesztő. Az iskolai tanulmányaink során is ezt a fejlesztői környezetet használtuk, így ezt ismertük a legjobban.
- Git és GitHub: A projekt végzése során létrejött dokumentumok, fájlok és kódok tárolására használtuk, a verziókezelésnek köszönhetően folyamatosan nyomon tudtuk követni a változásokat. Ezeknek köszönhetően bárhol és bármikor hozzá tudtunk férni könnyen a munkánkhoz. A projekt fejlesztése során végig egy main branch-et (fő ágat) használtunk.
- Postman: Professzionális API fejlesztő és tesztelő környezet, amellyel a frontend elkészülte előtt vizsgáltuk a végpontok HTTP válaszait.
- XAMPP: Lokális fejlesztői környezet, amely az Apache webszervert és a MySQL (MariaDB) adatbázist fogja össze egyetlen könnyen kezelhető felületen. Szintén a tanulmányaink során használtuk, ezért ezt találtuk megfelelőnek.

3.2 Csoportmunkát segítő szoftverek:

- **Trello**: Projektszervezéshez használt eszköz, így ez egy megfelelő lehetőséget nyújtott nekünk a projekt során elvégzendő tevékenységeinkhez. Felosztottuk, hogy ki mit fog csinálni, kijelöltünk közös, illetve egyéni feladatokat. Folyamatosan frissítettük a feladatok állapotát, újakat írtunk. Valamint itt vezettük a fontosabb Discord megbeszéléseket is, ahol valamilyen jellegű elakadás vagy félreértést beszéltünk meg.
- **Discord és Messenger:** Ha bármi egyéb jellegű kommunikációt kellett folytatnunk, akkor ezt a két platformot használtuk. A Discord szerverünkön folytattunk videó hívást, ahol a képernyőnket megosztva mutattuk egymásnak az aktuális állást (pl. amikor valamelyikünk beteg volt). Messengeren az egyéb megbeszélések történtek, hogy menjünk Discord-ra, valaki elakadt és segítségre van szüksége, vagy egymás emlékeztetése a feladatról.

3.3 Dokumentációkészítéshez használt eszközök:

- Microsoft Word: a részletes dokumentáció elkészítéséhez használtuk
- GitHub README.md: egy rövidebb dokumentáció, amely a GitHub Repository-ban egyből látható

3.4 Képtervezéshez és szerkesztéshez használt szoftverek

- Canva: a logó és banner elkészítéséhez használtuk

- Adobe Photoshop: a Canva-ban elkészített több logóból csináltunk egy véglegeset, ami itt látható

3.5 Backend Technológiák és Keretrendszerek:

- Node.js (v22+): Ez a szoftverfuttatási környezet az esemény vezérelt és aszinkron (nem blokkoló) működésének köszönhetően tökéletes választás egy olyan adatintenzív webalkalmazáshoz, mint a ClanVerse. Kialakítása lehetővé teszi rengeteg egyidejű felhasználói kapcsolat gyors kezelését, miközben a szerver memóriafelhasználása minimális marad
- Express.js (v5.1.0): Minimalista és rendkívül gyors webes keretrendszer a Node.js-hez. A REST API végpontok felépítésére és a kérés-válasz (Request-Response) ciklust módosító middleware-ek láncolására szolgál.
- MariaDB (v3.4.5+): Nyílt forráskódú adatbázis-kezelő rendszer. Azért esett rá a választásunk, mert az adatokat rendszerezve, átlátható táblázatokban tárolja. Lehetővé teszi, hogy a táblák között erős kapcsolatokat hozzunk létre (Foreign Keys, CASCADE), és biztosítja, hogy ne maradjanak hibás adatok a rendszerben.
- Sequelize (v6.37.7): Egy magas szintű, ígéret-alapú (Promise-based) Node.js ORM (Object-Relational Mapper) eszköz. Lehetővé teszi, hogy nyers SQL kódok helyett objektumorientált JavaScript kóddal manipuláljuk az adatbázist, jelentősen növelve a kód olvashatóságát és automatikusan kivédve az SQL Injection támadásokat a paraméterezett lekérdezésekkel.
- JSON Web Token (JWT v9.0.2): Állapotmentes hitelesítési ipari szabvány. A szerver nem tárol session adatokat a memóriában, hanem a kliens egy digitálisan aláírt token-t küld minden kérésnél, amely tartalmazza a felhasználó adatait és jogosultságait.
- Bcrypt (v6.0.0): Jelszavak biztonságos tárolására használt algoritmus. Ahelyett, hogy a felhasználók jelszavait sima szövegként mentené el, egy visszafejthetetlen, egyedi karaktersorozatot (úgynevezett "sózott hash"-t) készít belőlük. Ez garantálja a maximális adatbiztonságot: még egy esetleges szerveroldali adatszivárgás vagy adatbázis-feltörés esetén sem tudják elolvasni a jelszavakat.
- Multer (v2.0.2): Fejlett middleware a "multipart/form-data" formátumú kérések kezelésére. Kizárólagos feladata a felhasználók profilképeinek és a játékok logóinak biztonságos, memóriakorlátos szerverre töltése és átnevezése.

3.6 Frontend Technológiák:

- Angular (v20.3): A Google által fejlesztett, komponens-alapú, vállalati szintű frontend keretrendszer. A Single Page Application (SPA) architektúra biztosítja, hogy az oldal újratöltés nélkül, dinamikusan cserélje a nézeteket a DOM manipuláció segítségével.
- TypeScript (v5.9.2): A JavaScript szigorúan tipizált (strongly typed) szuperszetje. Használata mind a frontend, mind a backend fejlesztés során drasztikusan csökkenti a futásidejű (runtime) hibák számát azáltal, hogy fordítási időben (compile time) fogja meg a típusinkonzisztenciákat.

- Bootstrap (v5.3.8): Globálisan elterjedt, mobil-első (mobile-first) CSS keretrendszer. Használatával a UI elemek reszponzívak, így a ClanVerse tableteken és okostelefonokon is vizuális kompromisszumok nélkül használható.
- RxJS (v7.8.0): A reaktív programozást teszi lehetővé. Az Angular ezen keresztül kezeli a HTTP kérések aszinkron adatfolyamait (Observable minta), ami felülmúlja a hagyományos Promise-alapú adatkötéseket.

IV. Felhasználói dokumentáció

4.1. Rendszerkövetelmények:

Az alábbiakban részletezzük a projekt futtatásához szükséges minimális és ajánlott rendszerkövetelményeket. A zökkenőmentes felhasználói élmény érdekében érdemes ezeket ellenőrizni a webalkalmazás elindítása előtt.

Fontos megjegyezni, hogy a projekt letöltéséhez (klónozáshoz), a csomagok telepítéséhez, valamint az alkalmazás több eszközről történő hálózati eléréséhez aktív és stabil internetkapcsolat szükséges. Ebben az esetben az eszközöknek azonos hálózaton kell lenniük! Ugyanakkor, ha a telepítés után az alkalmazást csak egyetlen eszközön (asztali számítógépen vagy laptopon), lokálisan (localhost) kívánjuk futtatni, ahhoz már nem elvárás az internetelérés.

Minimális

- Operációs rendszer: Windows 10, macOS 10.15, vagy Linux (Ubuntu 20.04 LTS vagy újabb)
- Memória: 2 GB RAM (Az Express keretrendszer és a MariaDB együttes, lokális futtatásához elegendő)
- Tárhely: 500 MB szabad hely (a forráskódnak, a "node_modules" könyvtárnak, és a feltöltött képek/logók átmeneti tárolásának)
- Node.js: 22.3 vagy újabb

Ajánlott

- Operációs rendszer: Windows 11, macOS 12, vagy Linux (Ubuntu 20.04 LTS vagy újabb)
- Memória: 4 GB RAM vagy több
- Tárhely: 1 GB+ SSD
- Node.js: 22.19 vagy újabb

4.2. Előfeltételek:

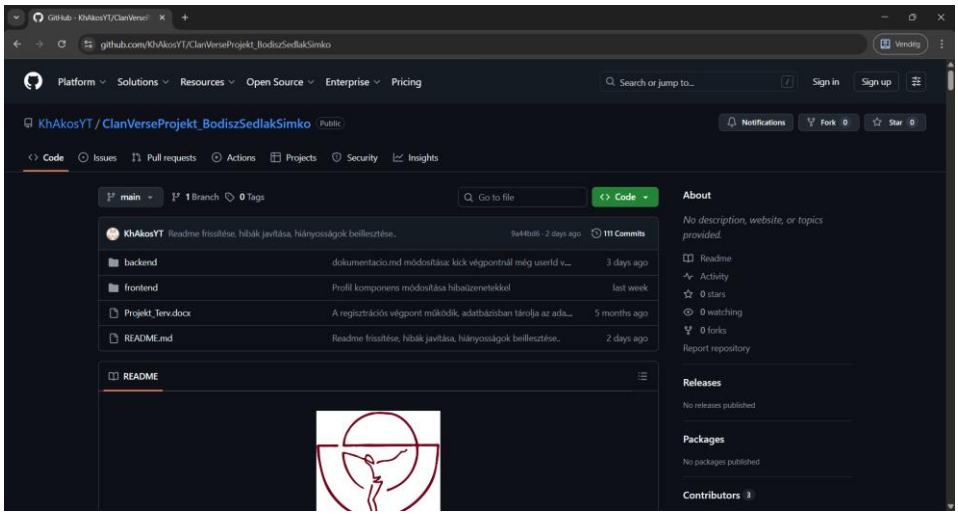
Győződj meg róla, hogy telepítve van:

- Node.js (22.0.0 vagy újabb) - https://nodejs.org/en/download
- XAMPP Control Panel (8.2 vagy újabb) - https://www.apachefriends.org/hu/index.html
- Visual Studio Code - https://code.visualstudio.com/
- Github desktop - https://desktop.github.com/download/
- vagy Github oldal - https://github.com/KhAkosYT/ClanVerseProjekt_BodiszSedlakSimko
- Windows 10 esetében WinRAR: https://www.winrar.hu/downloads

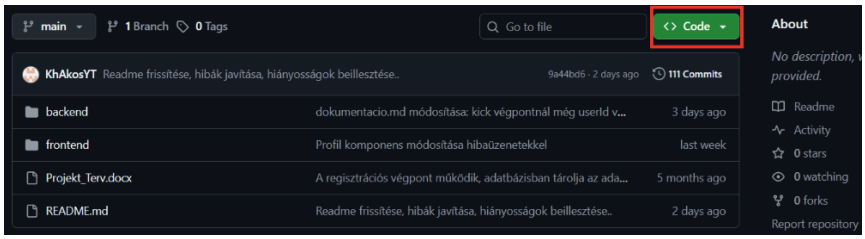
4.3. Projekt klónozása

4.3.1. Letöltés Github oldalról

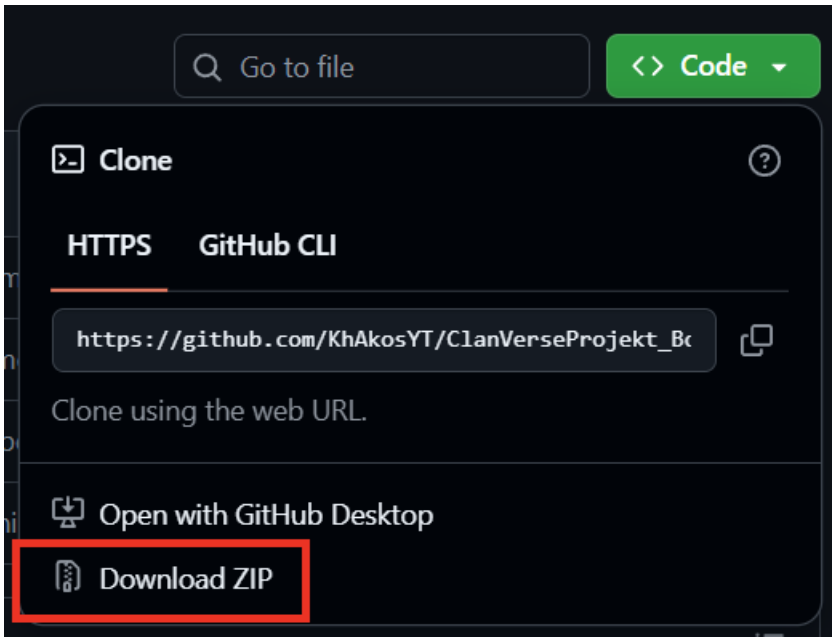
Nyisd meg Githubon a ClanVerseProjekt_BodiszSedlakSimko repository-t.



Nyomj a zöld ***„Code”*** gombra.



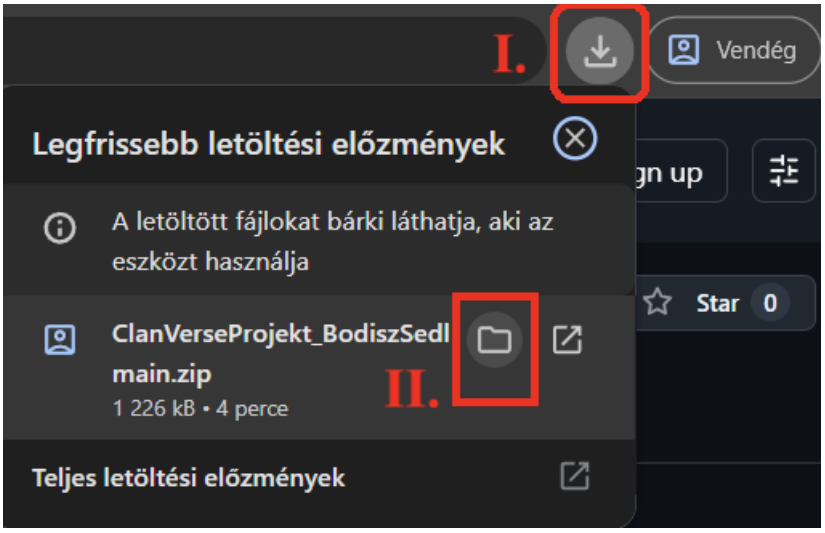
Itt nyomj a ***„Download ZIP”*** gombra.



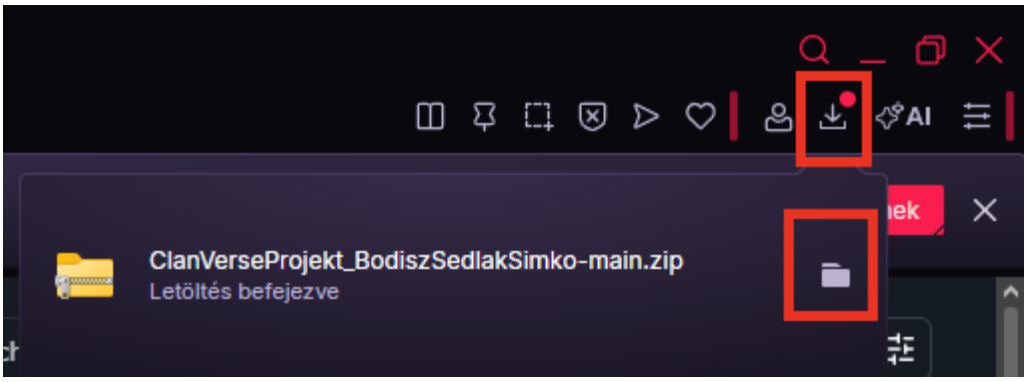
Ezután vagy automatikusan letölti a „letöltések” mappába, vagy **neked kell megadnod, hova töltsön le.**

Miután letöltött, nyomj a jobb a böngésződ jobb felső sarkában a letöltésekhez, majd „megjelenítés mappában”

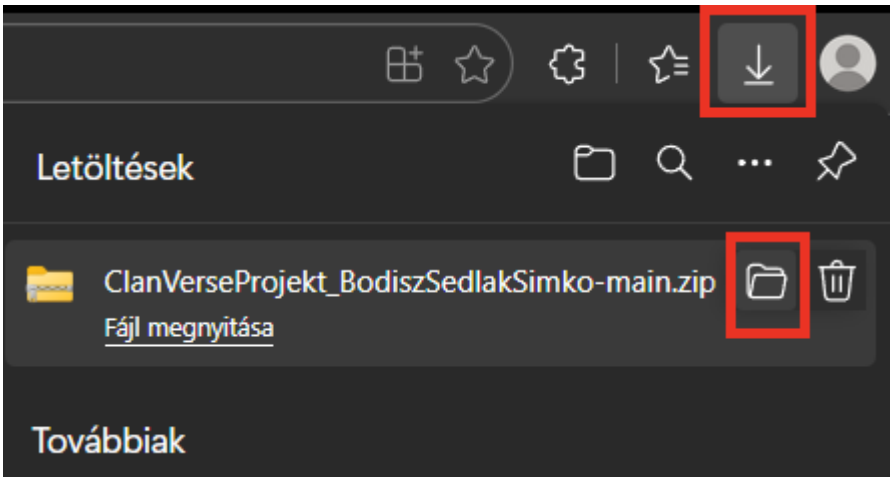
- Google Chrome:



- Opera GX:

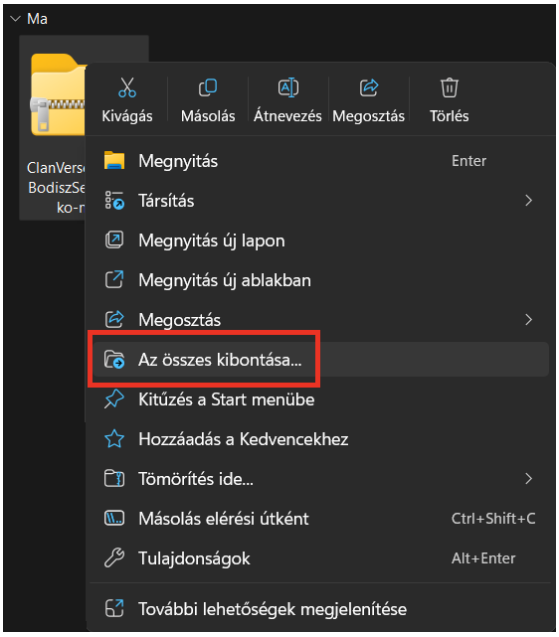


- Microsoft Edge:

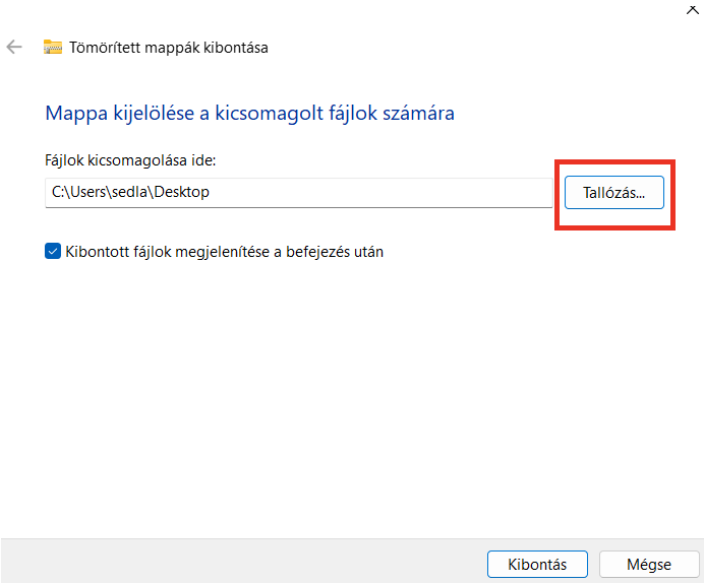


Ha sikerült megnyitni a mappában, akkor jobb klikkelni kell rá majd a kicsomagolás gombot kell megnyomni. Legegyszerűbb, ha az Asztalra csomagoljuk ki.

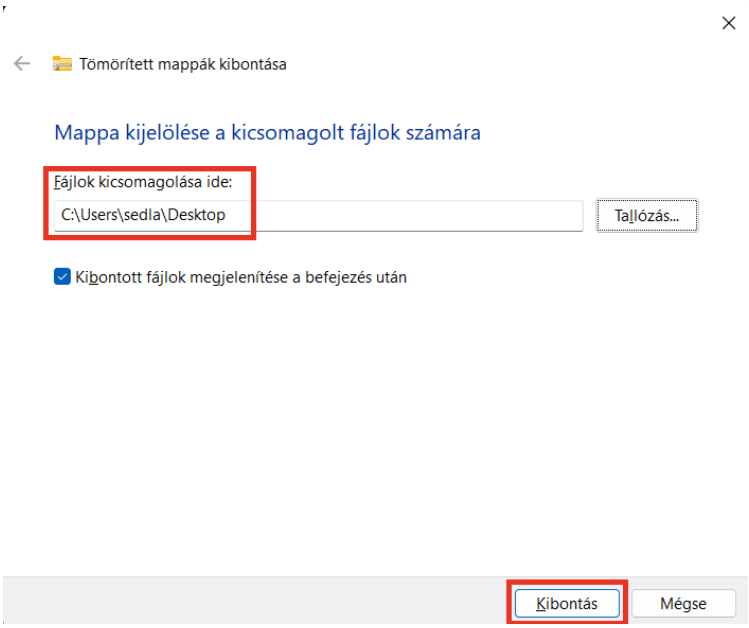
- Windows 11 esetében:



Ezután kiválaszthatjuk, hova csomagoljuk ki az állományt. Ezt a tallózás gombbal lehet megtenni.

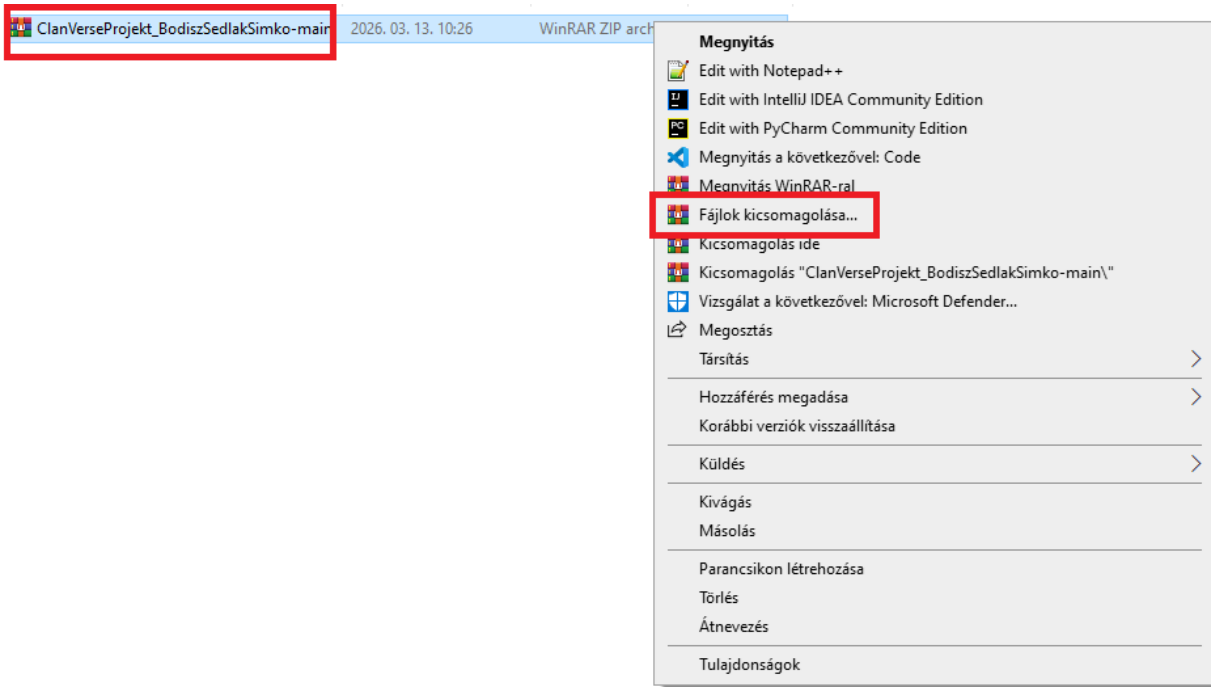


Én az asztalra szeretném kicsomagolni, mivel én megadtam az útvonalat, így rányomok a ***„Kibontás”*** gombra.

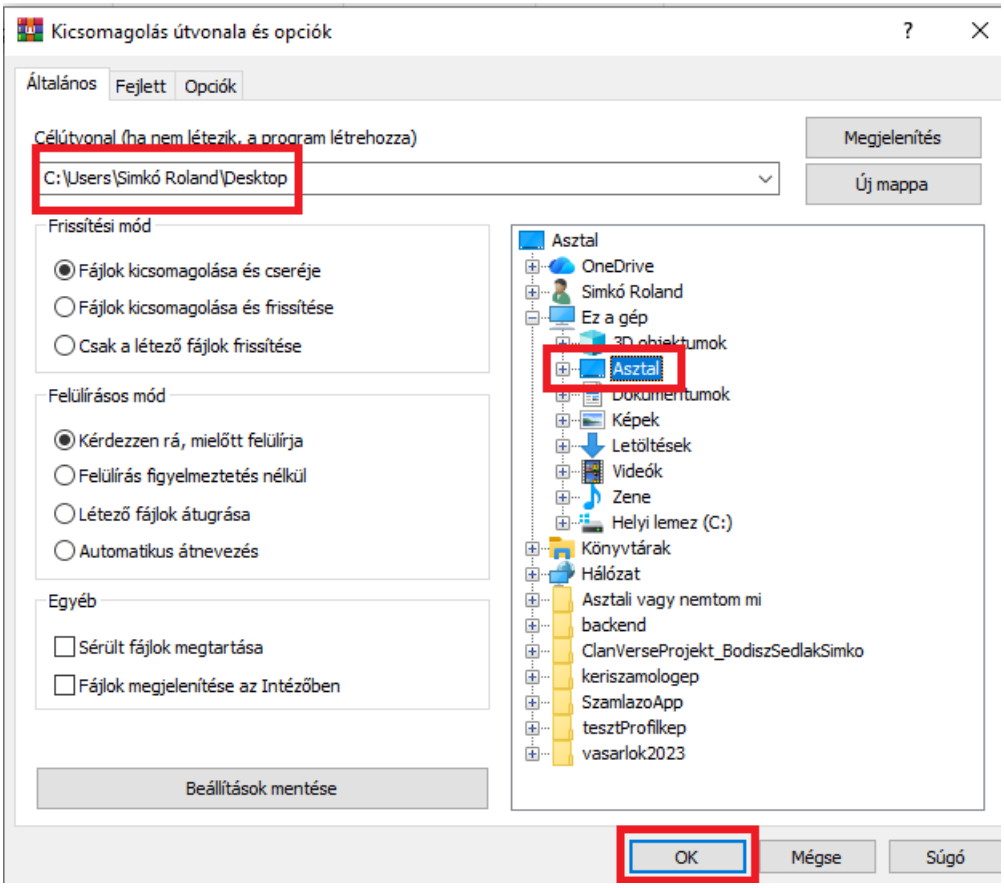


- Windows 10 esetében (WinRAR-al):

Jobb egérgombbal kattintsunk a tömörített állományra, majd nyomjunk a ***„Fájlok kicsomagolása”*** gombra.

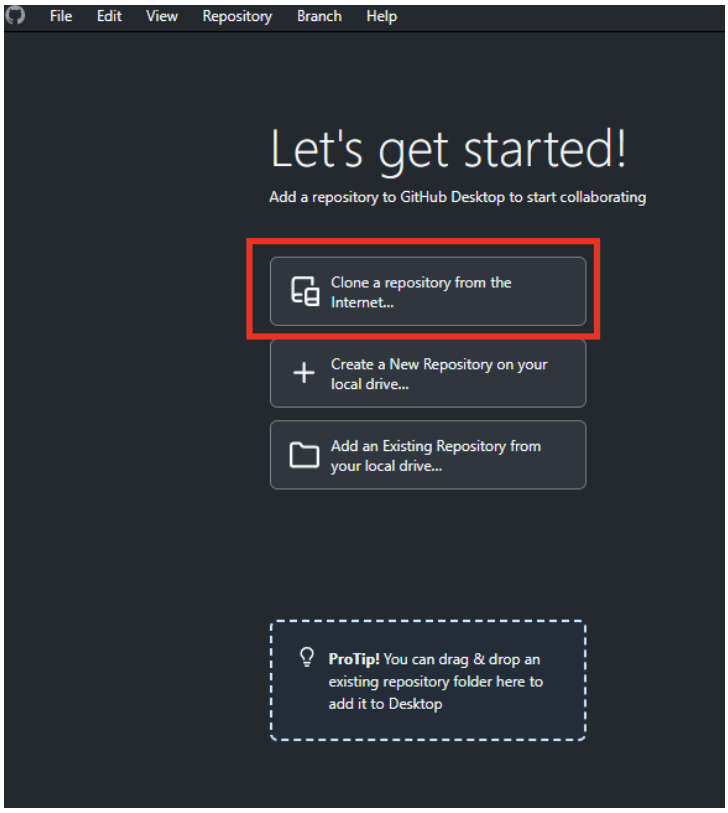


Ezután nyomjunk a felugró ablaknál az ***„Asztal”*** opcióra, majd az ***„Ok”*** gombra.

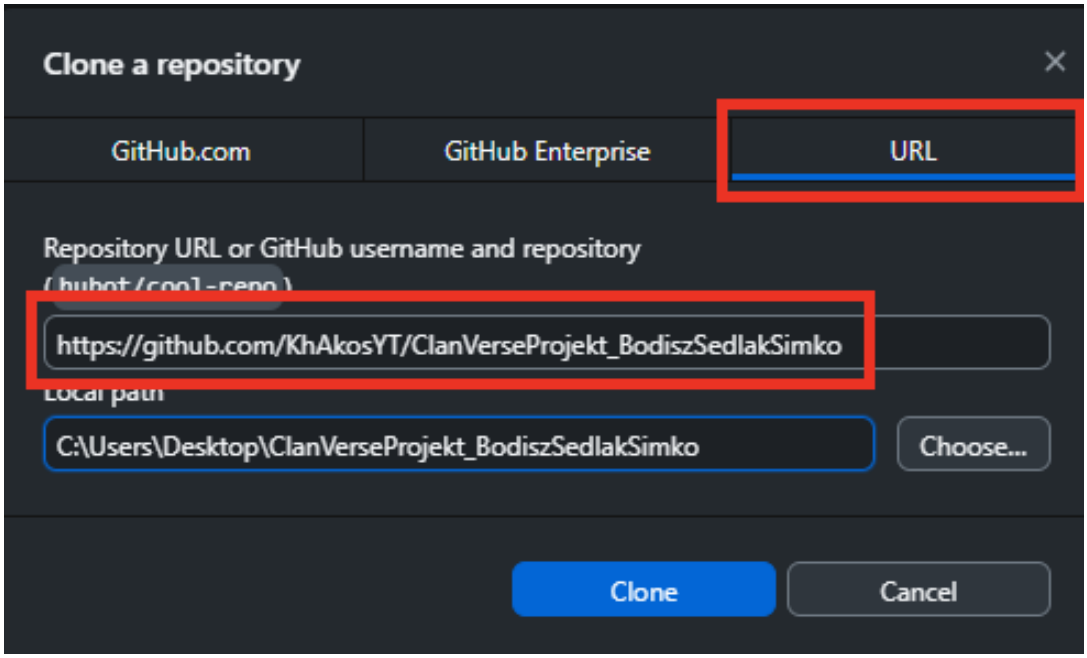


4.3.2. Letöltés Github Desktop alkalmazással

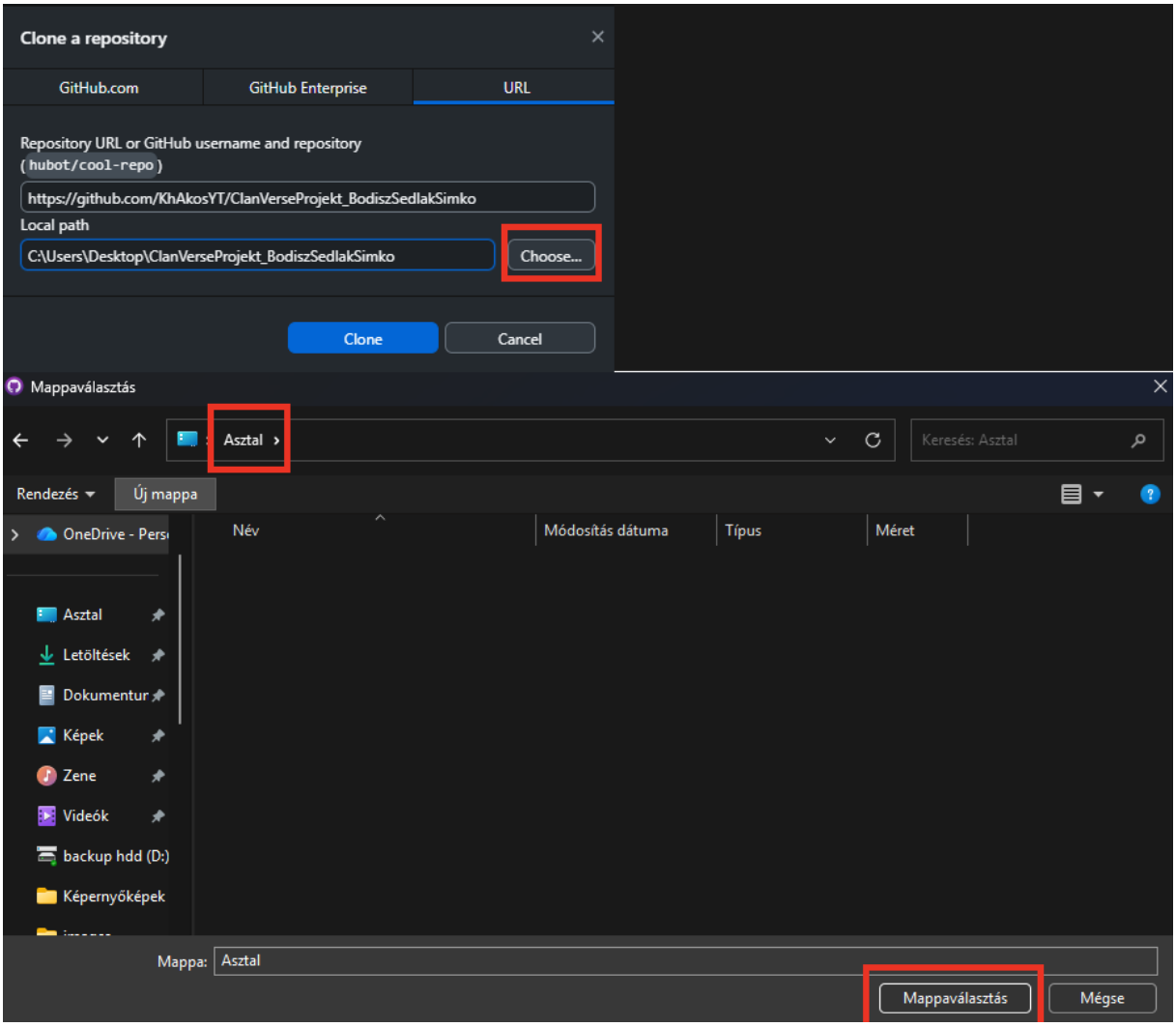
Nyissuk meg a Github Desktop alkalmazást, majd nyomjunk a „**Clone repository from the internet”** gombra.



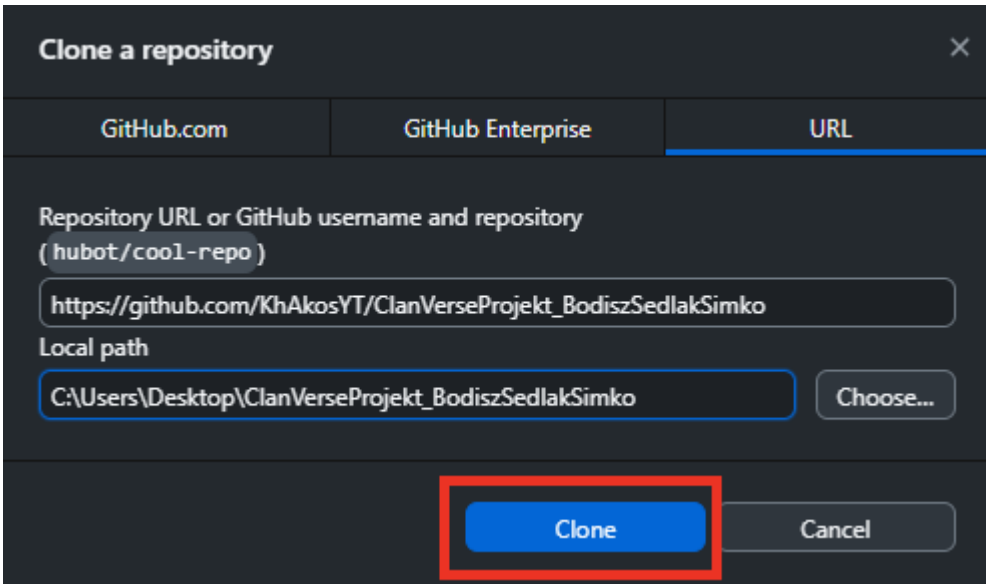
A bejelentkezés elkerülése érdekében nyomjunk az URL gombra, majd illesszük be a ClanVerse repo linkjét (https://github.com/KhAkosYT/ClanVerseProjekt_BodiszSedlakSimko)



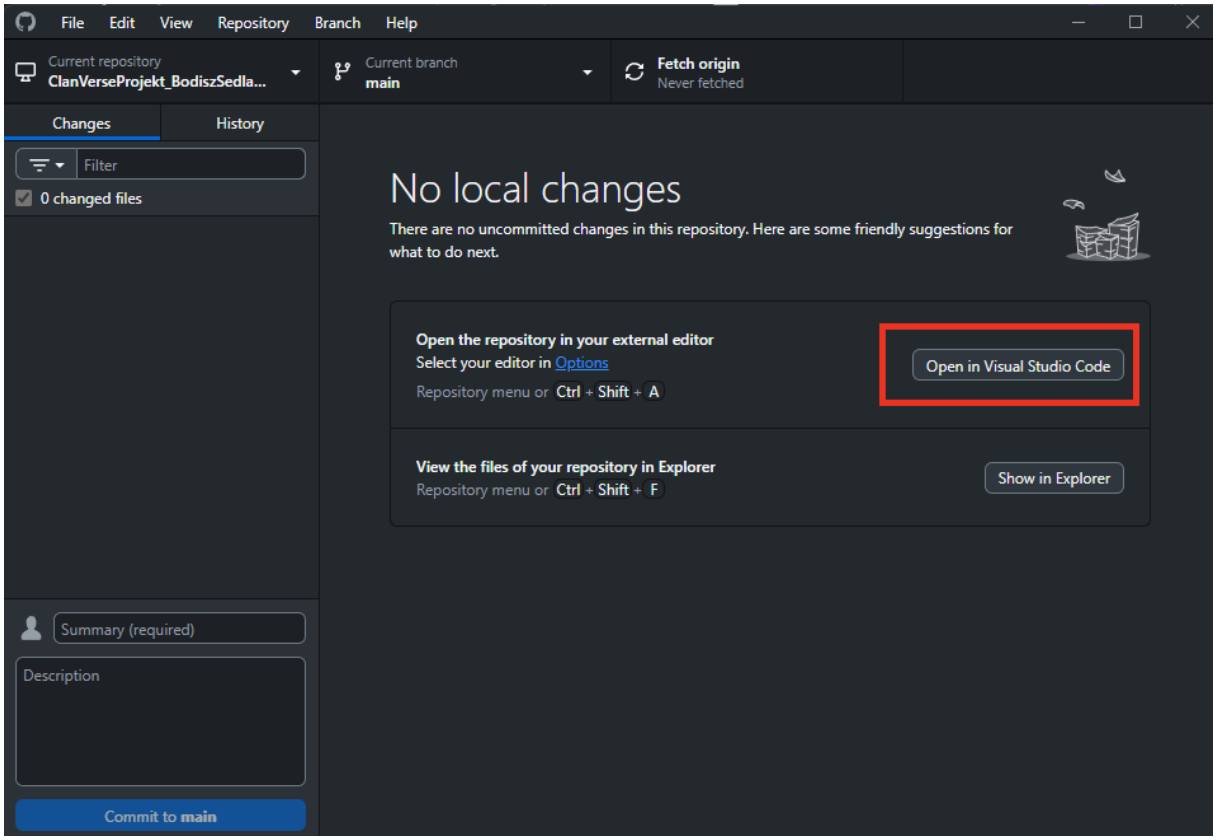
A local path résznél tudjuk megmondani, hogy hova szeretnénk letölteni a projektet. Én az asztalra telepítem.



Ha kiválasztottuk, nyomjunk a **Clone** gombra



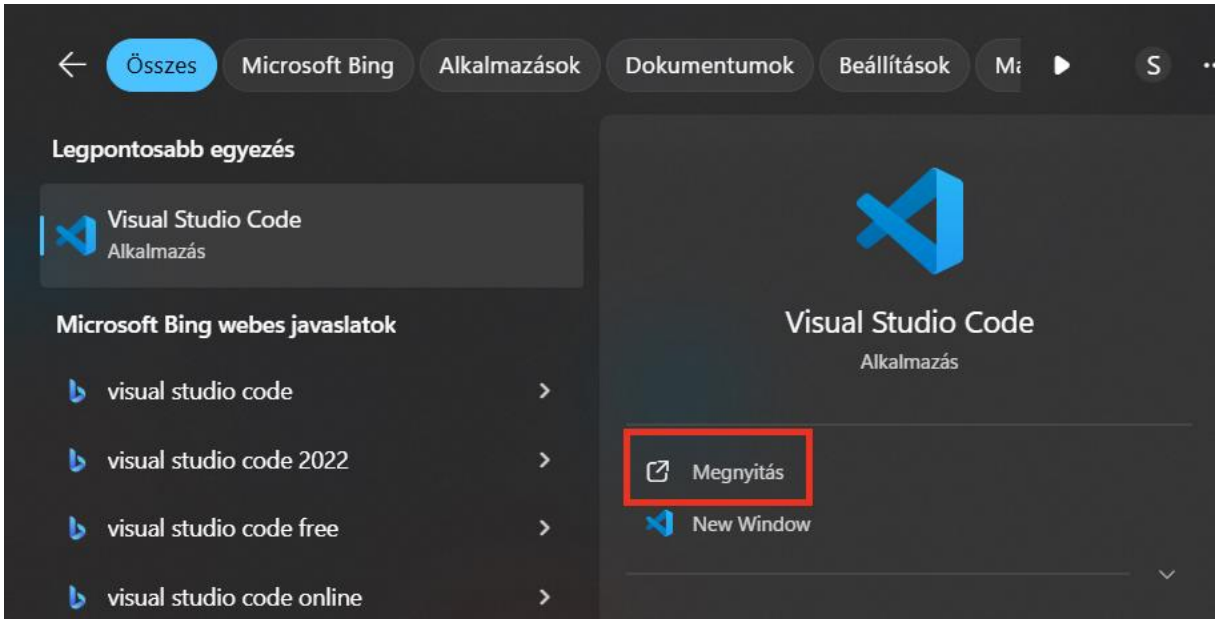
Klónozás után nyissuk meg Visual Studio Code-ban az alábbi gombbal, ezzel a **4.4-es pont átléphető**



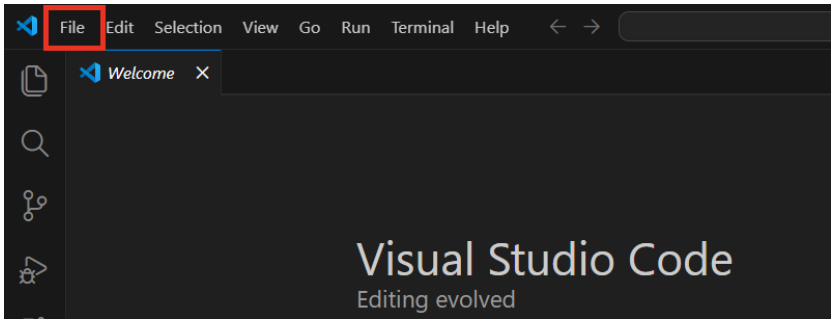
4.4. Projekt megnyitása Visual Studio Code-ban

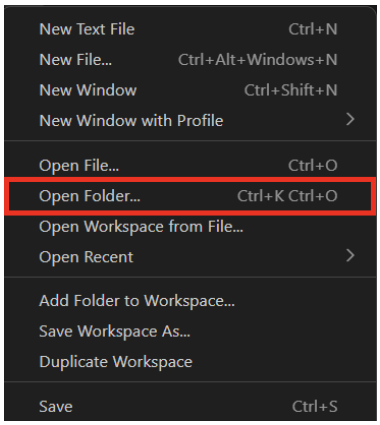
Elsőlépésként nyissuk meg a Visual Studio Code-ot.

Írjuk be, hogy Visual Studio Code, majd nyissunk meg

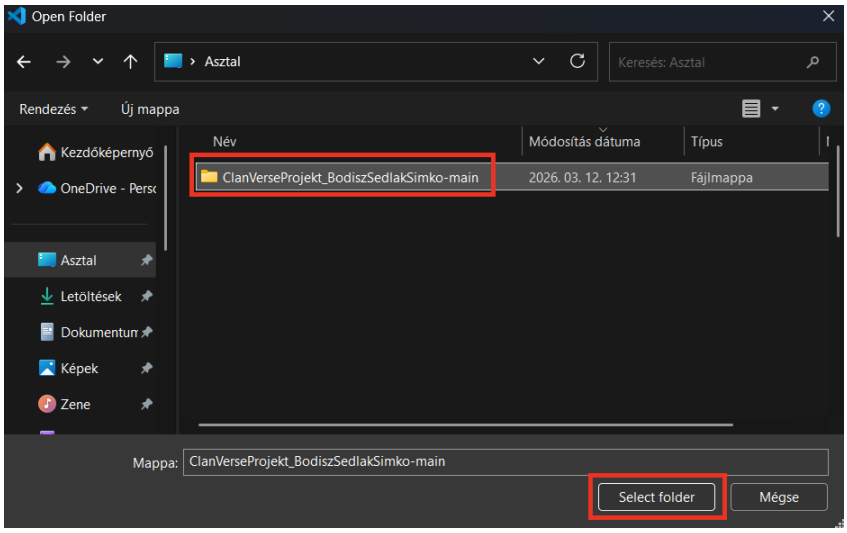


Ha megnyílt a VSC (Visual Studio Code), akkor nyissuk meg a kicsomagolt mappát. Kattintsunk a bal felső sarokban a **File** gombra, majd az **Open Folder** opcióra





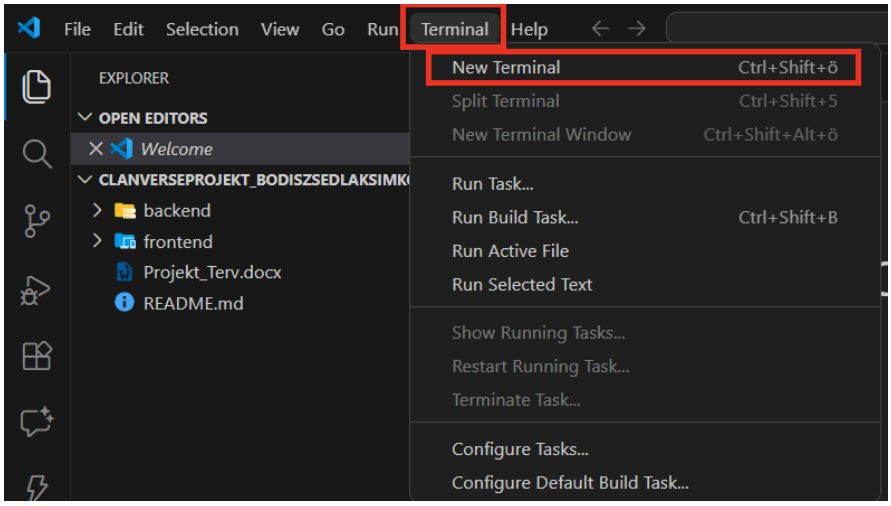
Itt keressük ki a letöltött mappát (ClanVerseProjekt_BodiszSedlakSimko-main) ott, ahova kicsomagoltuk. Rákattintunk és utána nyomjuk meg a „Select folder” gombot



4.5. Backend Beállítása

4.5.1 Parancssor megnyitása

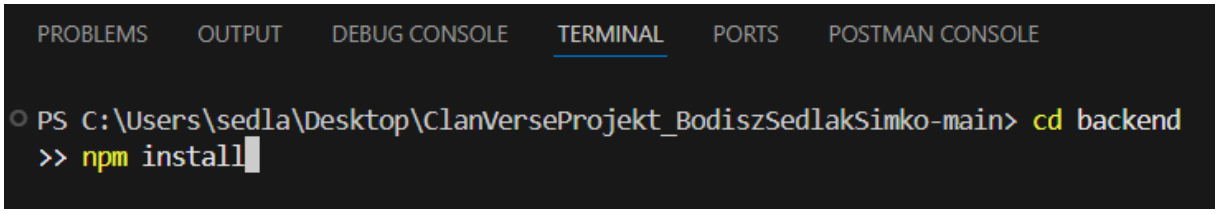
Visual Studio Code-ban nyissunk egy parancssort (terminal)



Ide másoljuk be az alábbi 2 parancsot majd nyomjunk *Enter*-t:

cd backend

npm install



Ha ezt a hibát írja a terminal

- npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see about_Execution_Polic

- ies at https:/go.microsoft.com/fwlink/?LinkID=135170.

- At line:1 char:1

- + npm i

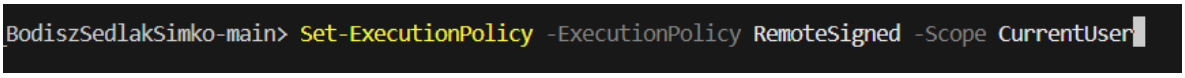
- + ~~~

- + CategoryInfo : SecurityError: (:) [], PSSecurityException

- + FullyQualifiedErrorId : UnauthorizedAccess

akkor az alábbi parancsot kell beírni ugyan ide:

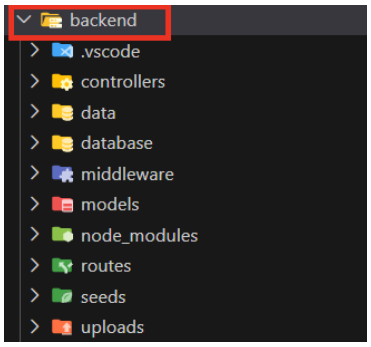
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser



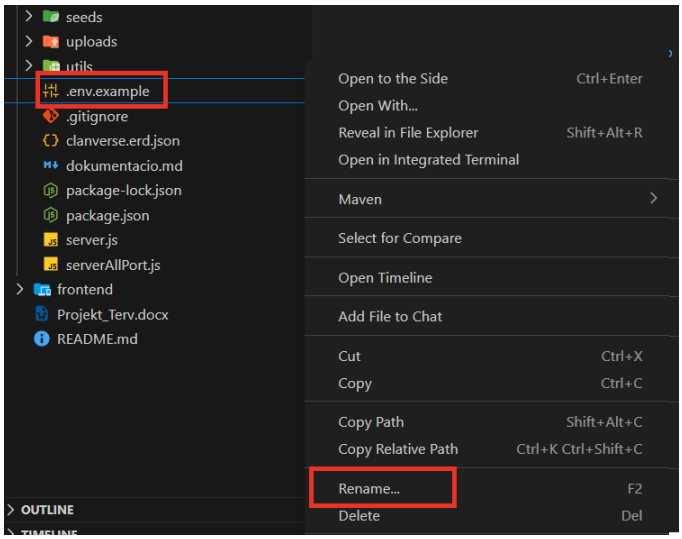
Ha beillesztetted a parancsot, ismét beírhatjuk az *npm install* parancsot, ezzel letöltöttük a csomagokat, amik szükségesek a backend-hez.

4.5.2 Környezeti változók beállítása

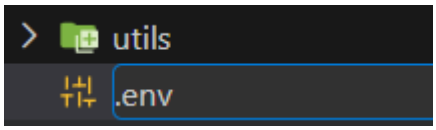
Visual Studio Code-ban bal oldalt nyissuk le a ***„backend”*** mappát



Jobb egérgombbal kattintsunk a ***„.env.example”*** fájlra, majd nyomjunk a ***„rename”*** gombra.

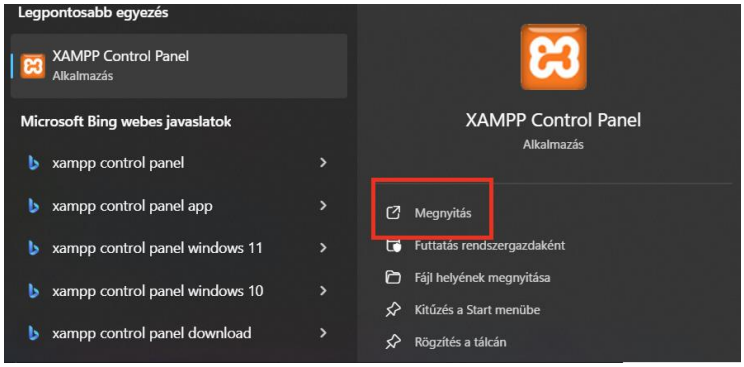


Töröljük ki a végéről a .example részt, majd nyomjunk Entert.

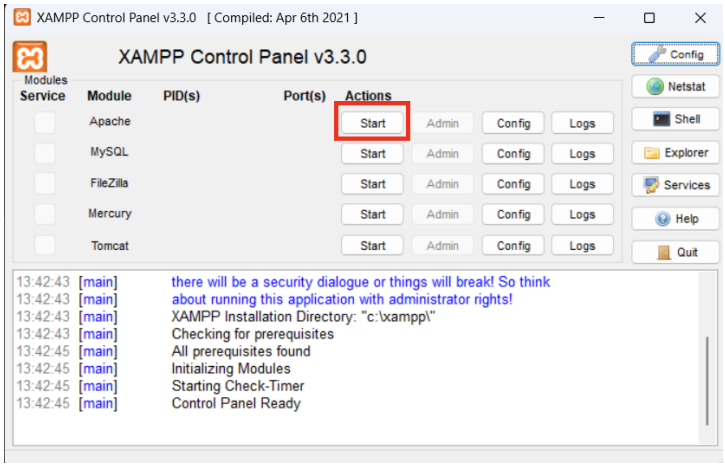


4.5.3 Adatbázis beállítása

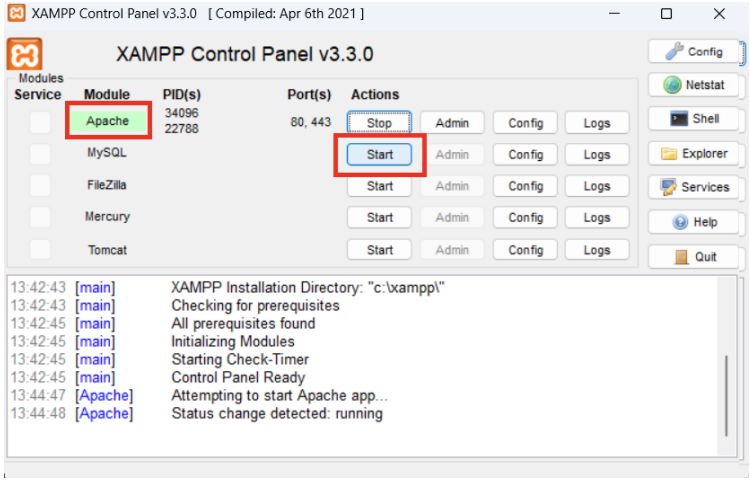
Nyissuk meg a XAMPP Control Panel-t.



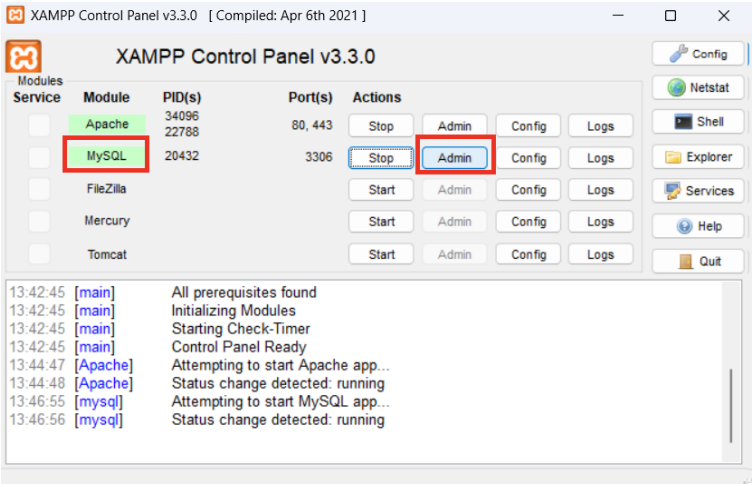
Ha megnyílt, nyomjunk az **Apache** melletti **Start** gombra.



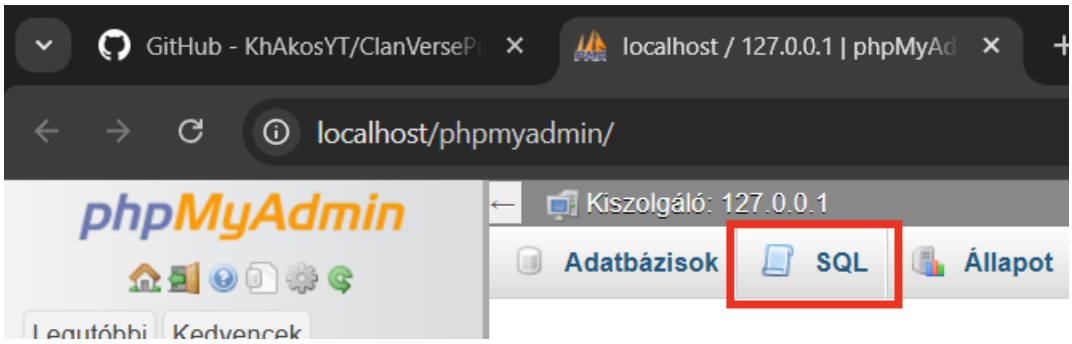
Várjuk meg, amíg zöld nem lesz, majd a MySQL-nél is nyomjuk meg a **Start** gombot.



Ha a MySQL is zöld, akkor nyomjunk az SQL **Admin** gombjára



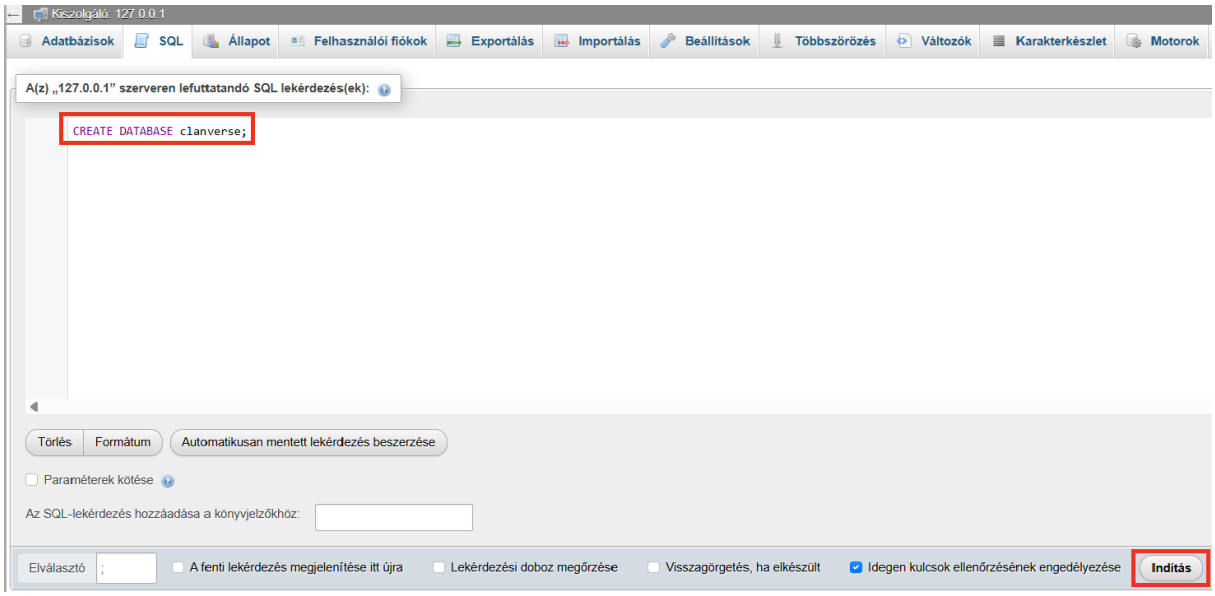
Az Admin gomb megnyomása után megnyitja a böngészőben a http://localhost/phpmyadmin/ címet. Itt nyomjunk az SQL gombra.



Az SQL gombra nyomva megnyílik az SQL kódfelülete. Ide írjuk be az alábbi SQL kódot:

CREATE DATABASE clanverse;

Majd a jobb alsó sarokban lévő **Indítás** gombbal futtassuk le a kódot



Ezzel létrehoztuk az adatbázisunkat

4.5.4 Backend szerver elindítása

Ha letelepítettük a csomagokat, létrehoztuk az adatbázist, akkor nincs más hátra, mind visszamenni a Visual Studio Code-ba, és elindítani a szervert.

Ezt az alábbi paranccsal lehet megtenni:

node server.js

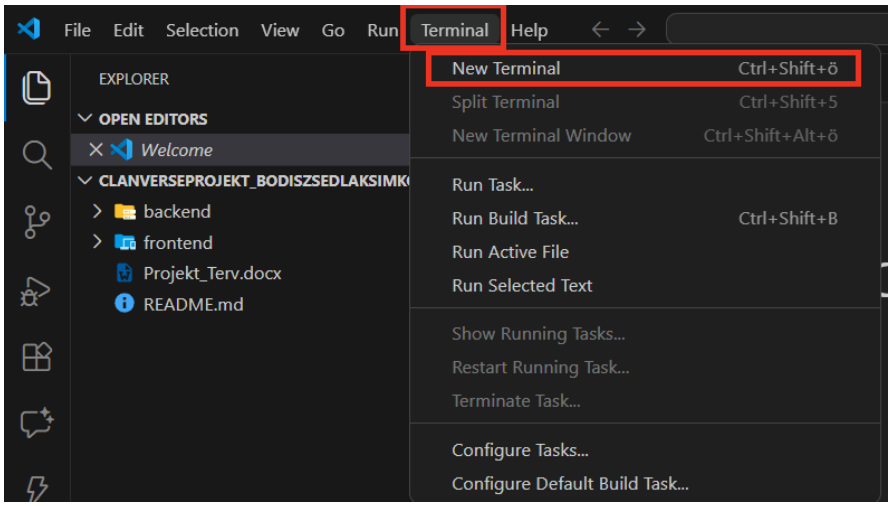
Ha minden jól ment, akkor a terminálon ennyinek kell megjelennie:

```
PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main\backend> node server.js
[dotenv@17.2.3] injecting env (13) from .env -- tip: suppress all logs with { quiet: true }
Szerver elindult! http://localhost:3000
Siker: clanverse
Az adatbázis szinkronizálva.
Sikeres feltöltés! (20 játék)
Sikeres feltöltés! (20 felhasználó)
Sikeres feltöltés! (10 klán)
Sikeres feltöltés! (38 klántag)
Sikeres feltöltés! (13 üzenet)
```

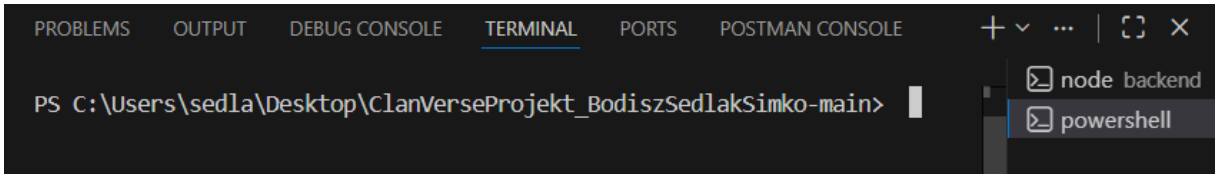
4.6 Frontend beállítása

4.6.1 Parancssor megnyitása

Ismét nyissunk egy terminált, ahogy a 4.5.1-es pontban tettük. A szerver terminálját ne zárja be!



Ha megnyitotta, akkor így kell kinéznie a lenti parancssornak



4.6.2 Angular és csomagok telepítése

Először telepítenünk kell az Angular megfelelő verzióját, amit megtehetünk itt a terminálon belül az itt látható paranccsal:

npm install -g @angular/cli@20.3

Ez egy kicsit hosszabb folyamat, amíg letölt. Ha letelepült az Angular, ezt kell látni a konzolon:

```
PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main>  npm install -g @angular/cli@20.3

added 18 packages, removed 70 packages, and changed 278 packages in 18s

66 packages are looking for funding
  run `npm fund` for details
PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main>
```

Ha ezt látja, akkor másoljuk be az alábbi 2 parancsot majd nyomjunk *Enter*-t:

cd frontend

npm install

Ezzel sikeresen telepítette az Angular-t és a futtatáshoz szükséges csomagokat.

4.6.3 Frontend elindítása

Ha települt minden frontend csomag, fut a backend, akkor nincs más hátra, mint az alábbi paranccsal elindítani az Angular-t:

ng serve

HA esetleg nem indulna így, akkor az ***npm start*** paranccsal is el lehet indítani. Ennek a módszernek a használatára egy példa: iskolában olyan gépen teszteltük, ahol nem volt telepítve az Angular, és a telepítéskor – mivel nincs rendszergazdai hozzáférésünk – nem települt az *npm* globális könyvtárába, így nem ismerte fel az *ng* parancsot

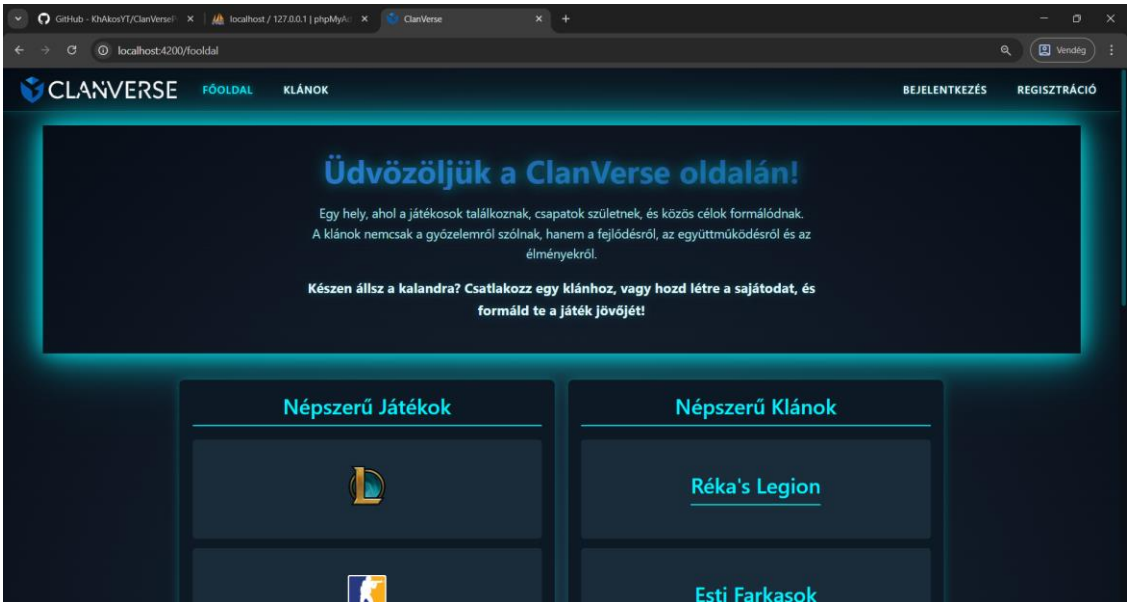
Ha mindent jól csináltunk, a konzolon kapott üzenetnek ez a vége:

```
PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main\frontend> ng serve
chunk-3EHPWYUN.js   | message        |  17.90 kB |
chunk-ZFR67GHC.js   | createclan     |  17.09 kB |
chunk-ZAMZQZWC.js   | aszf           |  16.06 kB |
chunk-3VOQQWER.js   | fooldal        |  13.80 kB |
chunk-5CSNBR2O.js   | login          |  13.23 kB |
chunk-QURAARHR.js   | admin          |  12.75 kB |
chunk-TW5BO567.js   | -              |   2.62 kB |

Application bundle generation complete. [7.114 seconds] - 2026-03-12T14:51:40.826Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
```

Ezzel sikeresen elindítottuk a projektet: a frontend a http://localhost:4200 címen elérhető.



4.7 Minden porton futó projekt beállítása

Ez a része arra szolgál, ha esetleg több eszközön szeretnénk tesztelni a webalkalmazást.

FONTOS megjegyezni: minden eszköznek ugyan azon a hálózaton kell lennie! Nem számít, hogy vezetékes vagy vezeték nélküli a kapcsolat, a lényeg, hogy „egy router-en legyenek” az eszközök!

4.7.1 Localhost leállítása:

Első lépésként a fenti leírás szerint futtassuk a projektet. Ha már fut, akkor az alábbi lépéseket kell megtenni:

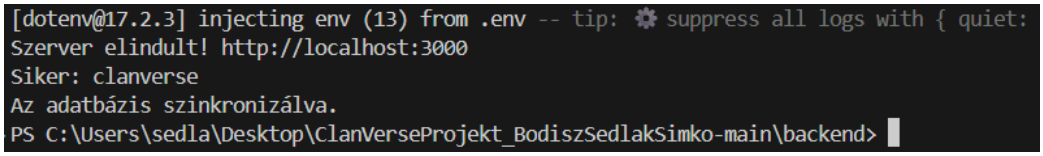
Állítsuk le mind a backend-et és a frontend-et. Ezt úgy tudjuk megtenni, hogy mind a két terminálba kattintva nyomunk egy ***Ctrl + C*** gombkombinációt. Ha ezt látjuk, akkor jól csináltuk.

```
Application bundle generation complete. [2.442 seconds] - 2026-03-12T15:21:42.931Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main\frontend> Ctrl + C után ismét írhatok ide
```

Ezt a lépést meg kell tenni a backenden is.

A backend termináljába lépve ismét nyomjuk meg a ***Ctrl + C*** gombkombinációt. Ezt kell látnunk:



Sikeresen leállítottuk a frontendet és a backendet

4.7.2 Minden porton figyelő szerver indítása:

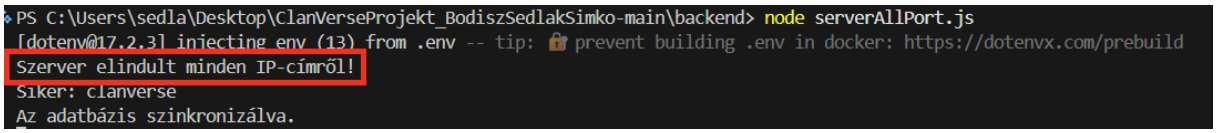
Hasonló lépéseket kell megtennünk azért, hogy a serverAllPort.js-t elindítsuk.

Amennyiben a backend mappában vagyunk, ezt a parancsot kell beírnunk.

node serverAllPort.js

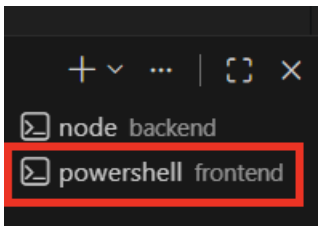
FONTOS: a XAMPP-ban a MySQL-nek továbbra is futnia kell, azt nem kell leállítani!

Ha ezt látjuk, sikeresen elindítottuk azt a szervert, ami figyel minden kérést.



4.7.3 Frontend konfigurálása és indítása:

Most ismét a terminálnál a frontendhez kell lépnünk.



Ha ott vagyunk, akkor be kell írnunk azt a parancsot, amivel lekérjük a számítógép IP címét. A parancs pedig az:

ipconfig

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main\frontend> ipconfig

Wireless LAN adapter Helyi kapcsolat* 10:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

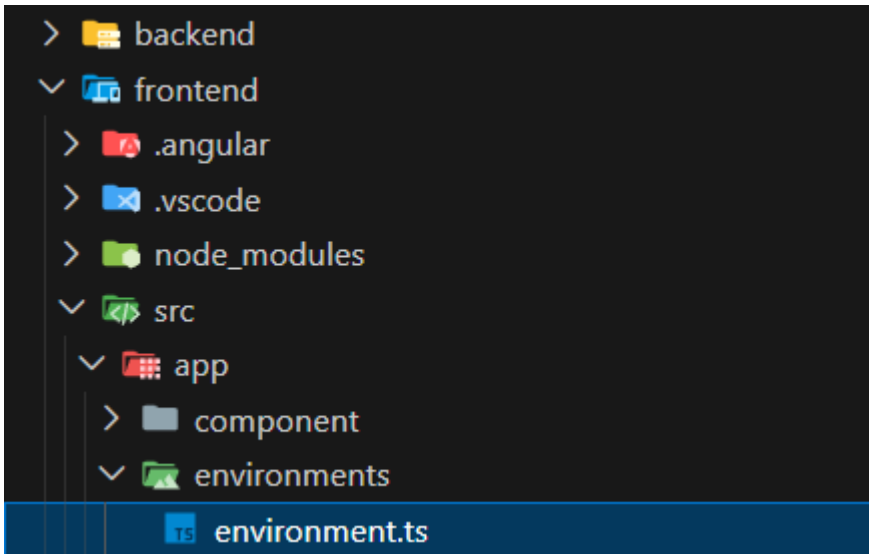
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : home
   IPv6 Address. . . . . . . . . . . : 2001:4c4e:268c:200:6f01:3c94:bfe2:4b0a
   Temporary IPv6 Address. . . . . . : 2001:4c4e:268c:200:e130:c325:7647:83f9
   Link-local IPv6 Address . . . . . : fe80::9386:adf:1a27:b3de%10
   IPv4 Address. . . . . . . . . . . : 192.168.1.104
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::8a0f:a2ff:fe23:6e2d%10
                                       192.168.1.1
PS C:\Users\sedla\Desktop\ClanVerseProjekt_BodiszSedlakSimko-main\frontend>
```

Itt az alján található meg a számítógép IP címe. Ahogy látható, az én címem a 192.168.1.104. Én a továbbiakban erre fogok hivatkozni, de neked a sajátodat kell használni.

Ha lekértük az IP címet, akkor meg kell keresnünk a frontendben az **environment.ts** fájlt.

```
ClanVerseProjekt_BodiszSedlakSimko/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── environments/
│   │   │   │   └── environment.ts    # Ebben a fájlban kell átírni
```



Az itt látható „apiUrl” és „serverUploadUrl” címében a két „localhost”-ot cserélni kell az IP címre.

Csere előtt:

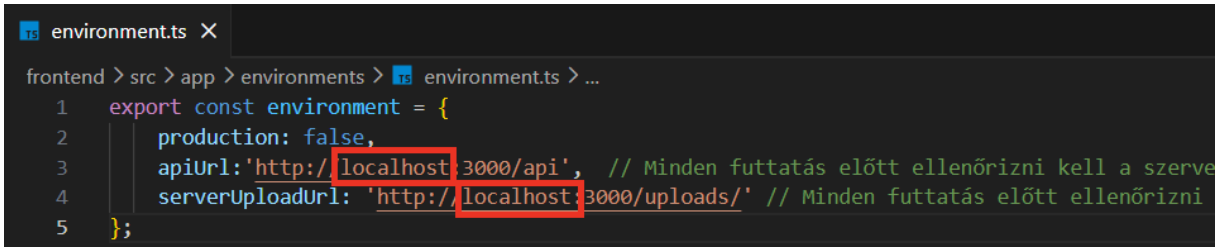
export const environment = {

production: false,

apiUrl:'http://localhost:3000/api',

serverUploadUrl: 'http://localhost:3000/uploads/'

};



Csere után:

export const environment = {

production: false,

apiUrl:'http://192.168.1.104:3000/api',

serverUploadUrl: 'http://192.168.1.104:3000/uploads/'

};

Fontos, hogy csak a „localhost” szót kell kicserélni. Ha bármi más hiányzik, nem fog működni!

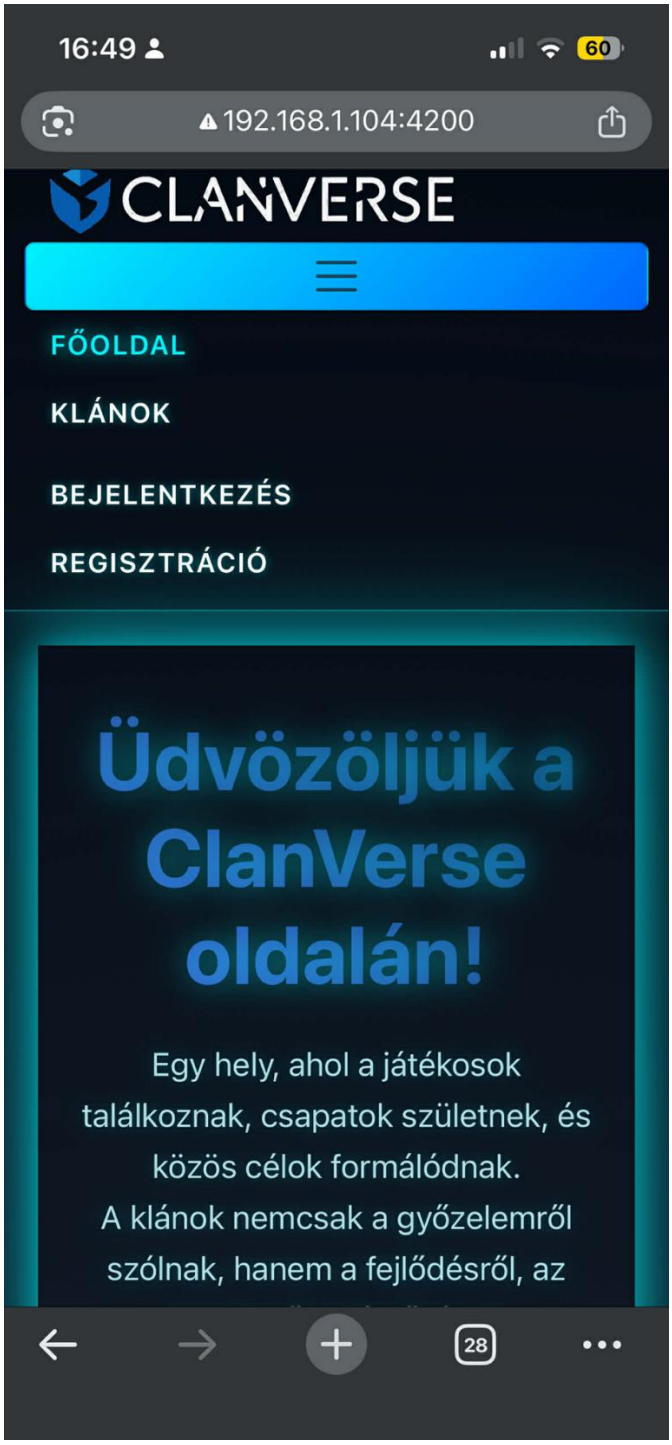
Ha átírtuk, akkor már csak el kell indítani az Angular-t ismét az alábbi paranccsal:

ng serve --host 0.0.0.0

Ha elindult, akkor, akkor a „**Network**” címen elérhető az alkalmazás minden ugyan azon a hálózaton lévő eszközről.

```
Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → Network: http://192.168.1.104:4200/
  → press h + enter to show help
```

Az én esetemben telefonról is meg tudom nyitni a http://192.168.1.104:4200/ címen. Ahogy látható ezen a képen, megnyitottam telefonról a ClanVerse oldalt.



V. Backend architektúra és üzleti logika

A backend architektúra szigorúan az MVC (Model-View-Controller) tervezési minta elveit követi, azzal a jelentős architekturális eltéréssel, hogy a "View" (Nézet) réteget itt egy teljesen különálló Angular kliensalkalmazás képviseli. A backend kizárólag egy REST API, amely JSON formátumú adatokat szolgáltat a hálózaton keresztül. Nincsenek szerveroldalon renderelt HTML sablonok (pl. EJS), így a szerver erőforrásai 100%-ban az üzleti logikára és az adatbázis-műveletekre fókuszálhatnak.

A rétegek és felelősségi köreik:

1. Router Réteg (Routes): Ez az alkalmazás kapuja. Fogadja a kliens bejövő HTTP kéréseit (GET, POST, PUT, DELETE metódusok), kiértékeli az URL végpontokat, és továbbítja a kérést a megfelelő köztesrétegnek (Middleware) vagy Vezérlőnek (Controller).
2. Middleware Réteg: A kérések előfeldolgozását, szűrését és validálását végzi még azelőtt, hogy a tényleges szerverlogika lefutna.
    - auth.js: Az azonosítás kulcsa. Ellenőrzi a HTTP fejlécekben (Headers) érkező JWT-t. Ha a token hiányzik, lejárt, vagy hamisított, a folyamat azonnal megáll, és 401 Unauthorized hibát küld vissza, hermetikusan elzárva a belső logikát az illetéktelen behatolóktól.
    - uploadPfp.js / uploadGameLogo.js: Ellenőrzi a hálózaton érkező bináris fájlokat. Kiszűri a nem kép formátumú fájlokat (biztonsági védelem kártékony kódok ellen), limitálja a fájlméretet, majd elmenti azokat a szerver lemezére.
3. Controller Réteg (Controllers): Az alkalmazás "agya". Itt történik a kérések paramétereinek (req.body, req.params, req.query) kinyerése, az üzleti szabályok kikényszerítése, a Modellek meghívása, és a megfelelő HTTP státuszkóddal (pl. 200 OK, 201 Created, 404 Not Found) rendelkező adatcsomag összeállítása.
4. Model Réteg (Models): A Sequelize ORM segítségével absztrahálja az adatbázist. Itt vannak definiálva a táblák oszlopai, az adattípusok, a default értékek és a relációs kapcsolatok (1:N, N:M).

VI. Adatbázis dokumentáció

A rendszer mögött álló relációs adatbázis a harmadik normálforma (3NF) követelményei szerint lett optimalizálva. Ez a kialakítás megszünteti a felesleges adatismétlődéseket, és megelőzi a hibákat az új adatok rögzítése, szerkesztése vagy törlése során. Az adatbázis neve a konfigurációkban: "clanverse". A karakterkódolás "utf8mb4", ami a kiterjesztett karakterkészletek teljes körű támogatását biztosítja.

A rendszer öt összekapcsolt fő táblára épül:

6.1. Users (Felhasználók) Tábla:

A platform lelke, az összes regisztrált felhasználó hitelesítési és profiladatait tartalmazza.

- id (INTEGER, Primary Key, Auto Increment): A felhasználó egyedi rendszerazonosítója.
- username (VARCHAR(255), Unique, Not Null): A megjelenített név, amelynek egyedinek kell lennie a könnyű azonosíthatóság és a klánokon belüli megkülönböztetés miatt.
- email (VARCHAR(255), Unique, Not Null): Hitelesítéshez és fiók-visszaállításhoz használt primer kulcsalternatíva.
- password (VARCHAR(255), Not Null): A Bcrypt algoritmussal generált irreverzibilis hash.
- profilePicture (VARCHAR(255), Not Null): A lemezen tárolt kép statikus útvonala. Default értéke egy alapértelmezett avatár ('profilePictures/default.png').
- isAdmin (BOOLEAN, Default false): Szuperfelhasználói jogosultságot (Admin panel elérése) biztosító flag.
- createdAt, updatedAt (DATETIME): A Sequelize által automatikusan menedzselt időbélyegek.

6.2. Games (Játékok) Tábla:

A platform által támogatott, adminisztrátorok által felvett játékok központi katalógusa.

- id (INTEGER, Primary Key, Auto Increment).
- gameName (VARCHAR(200), Unique, Not Null): A játék hivatalos megnevezése.
- logo (VARCHAR(255), Not Null): A játék ikonjának szerveroldali elérési útvonala. Ha az admin nem tölt fel fájlt, automatikusan kap egy Not Found képet logóként a játék. (gameLogos/gameNotFound.jpg)

6.3. Clans (Klánok) Tábla

A játékosok által alapított közösségek fő metaadatait tárolja.

- id (INTEGER, Primary Key, Auto Increment): a klán egyedi azonosítója

- name (VARCHAR(255), Unique, Not Null): A klán neve. A rendszer garantálja, hogy ne lehessen két azonos nevű csapatot alapítani.
- description (VARCHAR(1000), Nullable): A klán hosszabb, szabadon szerkeszthető bemutatkozó szövege.
- gameId (INTEGER, Foreign Key): Hivatkozás a Games tábla 'id' mezőjére (1:N kapcsolat, egy játéknak több klánja is lehet). Ha a játék törlődik, az adatbázis 'SET NULL' művelettel megőrzi a klánt, de megszünteti a játékkapcsolatot.
- ownerId (INTEGER, Foreign Key, Not Null): Hivatkozás a Users táblára. Megmutatja, ki az abszolút jogú alapítója a csapatnak.
- createrId (INTEGER, Foreign Key, Nullable): a klán létrehozójának az egyedi azonosítója. Ha törlődik a felhasználó, az az azonosító az adatbázisban marad.

6.4. ClanMembers (Klántagok) Tábla

Ez egy kiemelten fontos Kapcsolótábla, amely a Users és a Clans közötti Több-a-többhöz (N:M) kapcsolatot oldja fel strukturált formában.

- id (INTEGER, Primary Key).
- clanId (INTEGER, Foreign Key, Not Null): ON DELETE CASCADE szabállyal. Ha a klán megszűnik, az összes tagsági rekord azonnal, maradéktalanul megsemmisül.
- userId (INTEGER, Foreign Key, Not Null): ON DELETE CASCADE szabállyal. Ha törlődik a felhasználó, automatikusan törlődik a tagsági rekord.
- role (VARCHAR(100), Default 'member'): Ez határozza meg a klánon belüli hierarchiát. Az alapító 'leader' rangot kap, a csatlakozók 'member' rangot, ez alapján rajzolja ki a frontend a szerkesztési vagy törlési gombokat.

6.5. ClanMessages (Üzenetek) Tábla

A klánok privát chatfalának perzisztens tárolója.

- id (INTEGER, Primary Key).
- clanId (INTEGER, Foreign Key, Not Null): Meghatározza, melyik klán virtuális szobájában hangzott el az üzenet.
- userId (INTEGER, Foreign Key, Not Null): Az üzenet írója.
- text (TEXT, Not Null): Az üzenet tényleges, hosszú tartalma.
- createdAt (DATETIME): Az üzenet küldésének egzakt időpontja (A kliensoldalon ebből generálódik a Timestamp).

VII. Biztonsági megoldások és adatvédelem

A fejlesztés során kiemelt figyelmet fordítottunk a rendszer biztonságára. Az alkalmazást úgy építettük fel, hogy automatikusan kivédje a leggyakoribb webalapú támadásokat:

- Biztonságos jelszókezelés (Bcrypt): a jelszavakat sosem tároljuk olvasható nyílt szövegként. A Node.js-ben található Bcrypt csomag erre ad lehetőséget; visszafejthetetlen karaktersorozattá alakítja a jelszót. Ez lehetőséget ad arra, ha két felhasználó ugyanazt a jelszót adja meg, akkor sem azonos karaktersorozat lesz eltárolva.

- Biztonságos bejelentkezés és autentikáció (JWT): a JSON Web Token jelenleg az egyik legmodernebb módszer a munkamenet kezelések között. Ez egy digitálisan aláírt „belépőkártyát" biztosít a felhasználóknak és ha csak egy karaktert is kitöröl belőle, a szerver azonnal érvényteleníti és megtagadja a hozzáférést.

- CORS Policy: a Cross-Origin Resource Sharing házirend a szerveren (server.js) úgy lett beállítva, hogy alapértelmezetten csak és kizárólag a hivatalos frontend kliensről (pl. localhost:4200) érkező adatkéréseket fogadja be, eldobva minden más külsőforrásból származó http kérést. Az a szerver, amelyik minden porton fut és figyel (szerverAllPort.js), csak az egyszerűség kedvéért figyel mindenhol. Nem akartuk túlbonyolítani a telefon megnyitásának a konfigurációját, így ezen a szerveren a CORS minden kérést figyel.

- SQL Injection Védelem: megakadályoztuk azt a klasszikus támadási formát, amikor a beviteli mezőkön keresztül próbálnak kártékony utasításokat (például adattörlést) küldeni a szervernek. Rendszerünk szigorúan elkülöníti a felhasználó által beírt szöveget a tényleges programkódtól, így nem tudnak kártékony műveleteket és parancsokat lefuttatni.

- Ellenőrzött fájlfeltöltés: képfeltöltéskor a rendszer nemcsak a fájl kiterjesztését vizsgálja, hanem a HTTP kérésben érkező belső adatformátumot (MIME-típust) is. Ezzel jelentősen megnehezítjük annak a lehetőségét, hogy valaki képnek álcázott kártékony programot töltsön fel. Ráadásul biztonsági okokból minden feltöltött fájl egy automatikusan generált, új nevet kap, így kivédve az elérési utakkal való trükközést.

VIII. Részletes backend API dokumentáció

A backend kizárólag REST API-n keresztül kommunikál. A kommunikációs nyelv a JSON.

REST API URL konvenciói: https://restfulapi.net/resource-naming/

API Alap URL (Base URL): http://localhost:3000/api

8.1. Felhasználó Kezelés (/users)

Végpont URL	HTTP metódus	Leírás	Státuszkód / válasz
/register	POST	Új felhasználói fiók regisztrálása. A kérés végigfut az „uploadPfp" middleware-en is, így a regisztrációs adatok mellett (ultipart/form-data) profilkép is beküldhető. A jelszónak minimum 8 karakternek kell lennie. A jelszó titkosítva kerül be az adatbázisba	201 Created – A felhasználó sikeresen regisztrálva 400 Bad Request – Hiányzó vagy érvénytelen adat 409 Conflict – Felhasználónév vagy email már használatban van 500 Internal Server Error – Szerverhiba
/login	POST	Felhasználó hitelesítése. A Bcrypt ellenőrzi és összehasonlítja az adatbázisban lévő jelszót a bejelentkezéshez megadott jelszóval. Sikeres bejelentkezés után JWT-t küld a frontendnek.	200 OK – Sikeres bejelentkezés (JWT küldése) 401 Unauthorized – Hibás felhasználónév vagy jelszó 400 Bad Request – hiányzó adatok 500 Internal Server Error – Szerverhiba
/logout	POST	Kijelentkezés során használt végpont. Ha nincs token, nem tud a felhasználó kijelentkezni.	200 OK – Sikeres kijelentkezés 401 Unauthorized – Nincs autentikáció (token) 500 Internal Server Error – Szerverhiba
/validate	GET	Ellenőrzi a token hitelességét. Van-e, érvényes-e vagy lájárt.	200 OK – A token érvényes 401 Unauthorized – Érvénytelen vagy lejárt a token 500 Internal Server Error – Szerverhiba
/profile	GET	Bejelentkezett felhasználó profiladatainak lekérése. Az adatbázisban tárolt értékek elküldése (név, email, profilkép).	200 OK – Profil adatok 401 Unauthorized – Nincs autentikáció (token) 404 Not Found – Felhasználó nem található 500 Internal Server Error – Szerverhiba
/profile	PUT	Felhasználó profiladatainak módosítása (név, email, jelszó,	200 OK – Profil frissítve

		profilkép). Csak akkor tudja az adatait módosítani, ha a jelenlegi jelszót helyesen írta be. Ez extra biztonsági réteget ad a fióklopások ellen.	401 Unauthorized – Hibás jelszó, érvénytelen token 409 Conflict – Email vagy felhasználónév már használatban van 500 Internal Server Error – Szerverhiba

8.2. Klánok Kezelése (/clans)

Végpont URL	HTTP metódus	Leírás	Státuszkód / válasz
/	GET	Kilistázza az adatbázisban tárolt összes klánt. Csak klán nevet és játékcímet küld. Publikus végpont	200 OK- Összes klán adat 400 Bad Request – Hiányzó adat 401 Unauthorized – Nincs autentikáció 409 Conflict – Már létezik ilyen néven klán 500 Internal Server Error – Szerverhiba
/	POST	Új klán létrehozása. Az alapító token-jében tárolt userId automatikusan bekerül a klán adataiba (ownerId, createrId), valamint a klán tagok táblába „leader” jogosultsággal.	201 Created – Klán sikeresen létrehozva 400 Bad Request - Hiányzó adat 401 Unauthorized – Nincs autentikáció 500 Internal Server Error – Szerverhiba
/:id	GET	Specifikus klán adatainak lekérése: az URL param-ban megadott azonosítójú klán adatainak kilistázása. Csapat neve, leírása, játék neve és logója, klántagok és szerepkörük. Valamint kulcsfontosságú értékeket is küld, ami elősegíti a frontend dinamikusságát: a backend kiszámolja a lekérő felhasználó státuszát. Visszaadja, ha még nem tag, akkor a „canJoin” értéke TRUE, máskülönben FALSE. Ha a felhasználó a vezető, akkor küld egy „editable: true”-t.	200 OK – Klán adat + klán tagok + canJoin / editable 401 Unauthorized – Nincs autentikáció 404 Not Found – Nem található klán 500 Internal Server Error – Szerverhiba

/:id	PUT	Klán adatainak módosítása. Ezen a végponton lehet frissíteni a csapat információit. Csak a vezető (leader) fér hozzá.	200 OK – Klán frissítve 400 Bad Request – Érvénytelen adat 401 Unauthorized – Nincs autentikáció 403 Forbidden – Nincs módosítási joga 404 Not Found – Klán nem található 409 Conflict – Már létezik ilyen néven klán 500 Internal Server Error – Szerverhiba
/:id	DELETE	Megadott azonosítójú klán törlése az adatbázisból. A törlés után minden kapcsolata megszűnik és törlődik automatikusan (clanMembers és clanMessages táblákból).	200 OK – Klán törölve 401 Unauthorized – Nincs autentikáció 403 Forbidden – Nincs módosítási joga 404 Not Found – Klán nem található 500 Internal Server Error – Szerverhiba
/:id/join	POST	Csatlakozás a klánhoz. Kezeli a rendszer a tagságot: ha már a felhasználó tag, nem tud még egyszer csatlakozni.	200 OK – Sikeresen csatlakozás 401 Unauthorized – Nincs autentikáció 404 Not Found – Klán nem található 409 Conflict – Már tag vagy 500 Internal Server Error – Szerverhiba
/:id/leave	POST	Kilépés a klánból. Ha a csapat vezetője (leader) lép ki, akkor a legrégebbi tag (member) kapja meg a vezetői szerepet. Ha a vezetőn kívül nincs más csapattag, automatikusan törlődik a klán.	200 OK – Sikeresen kiléptél 401 Unauthorized – Nincs autentikáció 403 Forbidden – Nem vagy tagja a klánnak 404 Not Found – Klán nem található 500 Internal Server Error – Szerverhiba
/:id/kick/ :memberUserName	POST	Tag kizárása a klánból a neve alapján. Az URL param	200 OK – Tag kizárva

		tartalmazza a kizárni kívánt játékos nevét, az alapján kerül törlésre a ClanMembers táblából. Csak a vezető hajthatja végre ezt a feladatot. A rendszer ellenőrzi, hogy a kérést küldő tényleg „leader”-e. Saját magát nem tudja kizárni.	401 Unauthorized – Nincs autentikáció 403 Forbidden – Nincs kizárási joga 404 Not Found – Klán vagy tag nem található 500 Internal Server Error – Szerverhiba

8.3. Üzenetek és Chat (/messages)

Végpont URL	HTTP metódus	Leírás	Státuszkód / válasz
/:clanId	GET	A klán összes üzenetének lekérése. Szigorúan védett: a backend ellenőrzi, hogy a lekérő felhasználó tagja-e a megadott klánnak. Token alapján történő validálás	200 OK – Üzenetek és a hozzájuk tartozó felhasználó neve és küldési ideje 401 Unauthorized – Nincs autentikáció 403 Forbidden – Nincs jogosultága lekérni az üzeneteket 404 Not Found – Klán nem található 500 Internal Server Error – Szerverhiba
/:clanId	POST	Üzenet küldése a megadott klánba. Szintén szigorúan védett útvonal: csak tag küldhet üzenetet. Token alapján történő validálás	201 Created – üzenet elküldve 400 Bad Request- Hiányzó adatok 403 Forbidden – Nincs üzenetküldési joga 404 Not Found – Klán nem található 500 Internal Server Error – Szerverhiba

8.4. Játékok (/games)

Végpont URL	HTTP metódus	Leírás	Státuszkód / válasz
/	GET	Összes játék lekérdezése névvel és logó URL-el. Opcionális searc query paraméterrel név alapján lehet szűrni.	200 OK – Játékok listája 500 Internal Server Error – Szerverhiba

8.5 Index (/)

Végpont URL	HTTP metódus	Leírás	Státuszkód / válasz
/famous-games	GET	Népszerű játékok lekérdezése. A backend megszámolja hány klánnál vannak használatban a játékok és az alapján csökkenő sorrendben elküldi az első 5 játékot	200 OK – Népszerű játékok listája 500 Internal Server Error – Szerverhiba
/famous-clans	GET	Népszerű klánok lekérdezése. A backend megszámolja hány tagja van a klánoknak és az alapján csökkenő sorrendben elküldi az első 5 klánt	200 OK – Népszerű klánok listája 500 Internal Server Error – Szerverhiba

8.6 Admin (/admin)

Végpont URL	HTTP metódus	Leírás	Státuszkód / válasz
/is-admin	GET	Ellenőrzi, hogy a felhasználó admin-e. A tokenben el van tárolva. Az alapján küld a backend logikai értéket	200 OK – Admin státusz infó (true vagy false) 401 Unauthorized – Nincs autentikáció 500 Internal Server Error – Szerverhiba
/upload-game	POST	Új játék feltöltése. Az admin tud csak játékot hozzáadni a katalógushoz. Logót is fel tud tölteni, annak a nevét a játék nevéből alakítja át a backend. Nem kötelező játéklogót feltölteni, automatikusan kap egy „gameNotFound.jpg” képet	201 Created – Játék feltöltve 400 Bad Request – Hiányzó adat 401 Unauthorized – Nincs autentikáció 403 Forbidden – Nem admin 409 Conflict – Már létezik ilyen nevű játék 500 Internal Server Error – Szerverhiba

IX. Frontend Dokumentáció

9.1 Bevezetés

A ClanVerse projekt frontendje egy modern Angular alapú webalkalmazás, amely teljes funkcionalitást biztosít felhasználói klánok létrehozásához, kezeléséhez és az azoknak megfelelő üzenetcsatornák működtetéséhez. Az alkalmazás egy teljesen moduláris architektúrán alapul TypeScript-ben, amely lehetővé teszi az egyszerű kódkarbantartást, bővítést és hibakeresést. Az alkalmazás reszponzív Bootstrap 5 dizájnnal rendelkezik, amely az összes eszközön jól funkcionál.

9.2 Útvonalazás és navigáció

9.2.1 Az alkalmazás útvonalválasztása

Az Angular Router-en alapuló útvonalazás az app.routes.ts-ben van definiálva. Az alkalmazás korszerű lazy loading stratégiát alkalmaz, ahol az egyes komponensek csak akkor kerülnek betöltésre a böngészőbe, amikor azokra szükség van. Ez jelentősen csökkenti az alkalmazás indítási idejét és memóriahasználatát.

Az alkalmazás összes útvonala:

- " (root): Automatikus átirányítás a főoldal-ra
- /fooldal: A kezdőoldal, amely megjeleníti a népszerű játékokat és klánokat
- /registration: A felhasználó regisztrációs oldala új fiókok létrehozásához
- /login: Az autentikációs oldala a meglévő felhasználók számára
- /clans: Az összes klán megtekintésének fő oldala szűrési lehetőséggel
- /createclan: Új klánok létrehozásának oldala (védett authguard-dal)
- /message/:clanId: Klánspecifikus üzenetcsatorna a clanId paraméterrel (védett)
- /profile: Felhasználói profiladatok szerkesztésének oldala (védett)
- /admin: Adminisztratív panel játékok feltöltéséhez (védett)
- /aszf: Általános Szerződési Feltételek statikus oldala

9.2.2. Guard-ok és autentikációs védelem

Az authGuard egy CanActivateFn típusú útvonalvédő, amely az alkalmazás-specifikus védelmet biztosítja. A UserService-t használja a felhasználó JWT token-jének validálásához. Ha a token érvénytelen vagy lejárt, akkor az Observable-ből false értéket ad vissza, amely megakadályozza az útvonal aktiválódását. Az authGuard gyakorlatilag az összes autentikációt igénylő útvonal előtt működik.

9.3 Hitelesítési rendszer

9.3.1 JWT –alapú hitelesítés:

Az alkalmazás JSON Web Token (JWT) alapú autentikációs rendszert implementál. A token egy Base64 kódolt karakterlánc, amely három részből áll: header, payload és signature. A sikeres bejelentkezés után a token a böngészőben a localStorage-ben tárolódik "token" kulcs alatt. Ez a token az összes ezt követő API kérésben küldődik az Authorization fejlécben.

9.3.2 HTTP Interceptor az automatikus token kezeléshez:

Az authInterceptor egy HttpInterceptorFn típusú interceptor, amely az alkalmazás indítási konfigurációjában van regisztrálva. Az interceptor minden HTTP kérés előtt megvizsgálja, hogy létezik-e token a localStorage-ben. Ha igen, akkor az Authorization fejlécet hozzáadja az alábbi formátumban: "Bearer [token]". Az interceptor valamint a válaszok error kezelésért is felelős: ha a backend 401-es hibát ad vissza (Unauthorized), az interceptor eltávolítja a token-t a localStorage-ből és megjeleníti az autentikációs modal-t a felhasználónak.

9.3.3 Felhasználói hitelesítési folyamat:

A felhasználó bejelentkezésekor a LoginComponent elküldi a felhasználónév-jelszó párost a backend-nek. A backend ellenőrzi az adatokat és ha helytelen, hibaüzenetet ad vissza. Ha helyes, a backend egy JWT token-t ad vissza. A LoginComponent ezt a token-t localStorage-ben tárolja el, majd átirányítja a felhasználót az alkalmazás főoldalára. Az alkalmazás belsejében az authGuard automatikusan ellenőrzi, hogy a felhasználó még bejelentkezve van-e a kezdeti token validálás révén.

9.4 Alkalmazás-wide szolgáltatások (services)

9.4.1 UserService – Felhasználó kezelési szolgáltatás:

A UserService az összes felhasználóhoz kapcsolódó operációt kezel. Függvényei az alábbiak:

- validateToken (): Egy GET kérést küld az /api/users/validate végpontra a backend-nek. Ez ellenőrzi, hogy az aktuális token még érvényes-e. A sikeres validálás után a showAuthModal signal false-ra van állítva.

- register(userData: FormData): POST kérést küld az /api/users/register végpontra. A userData FormData, amely felhasználónév, email, jelszó és profilkép bináris adatait tartalmazza.

- login(loginData: any): POST kérés az /api/users/login végpontra. A bejelentkezéskor a komponens a felhasználónevet és jelszót küldi. A sikeres válasz tartalmazza a JWT token-t.

- logout(): POST kérést küld az /api/users/logout végpontra az alkalmazás szerver oldali session-t lejáratja.

- getProfile(): GET kérést küld az /api/users/profile végpontra az autentifikált felhasználó adatainak lekérésére.

- updateProfile(userData: FormData): PUT kérést küld az /api/users/profile végpontra a profil adatainak frissítésére (név, email, jelszó, profilkép).

Ezen túlmenően a UserService tartalmazza a showAuthModal signal-t, amely egy Angular Signal és az autentikációs modal megjelenítésének állapotát kezeli.

9.4.2 ClanService – Klán kezelési szolgáltatás:

A ClanService az összes klánhoz kapcsolódó CRUD operációt biztosítja:

- createClan(clanData, token): POST kérés az /api/clans végpontra. A clanData objektum tartalmazza a klán nevet, játék ID-t és leírást.

- getClans(gameId): GET kérést küld az /api/clans végpontra. Opcionálisan szűrheti az eredményeket játék ID alapján.

- getClan(id, token): GET kérés az /api/clans/[id] végpontra. Egy specifikus klán teljes adatait kéri le, beleértve a tagok listáját.

- deleteClan(id, token): DELETE kérés az /api/clans/[id] végpontra. A klántulajdonos tudja a klánokat törölni.

- updateClan(id, clanData, token): PUT kérés az /api/clans/[id] végpontra. A klán nevét, leírást lehet módosítani.

- joinClan(id, token): POST kérés az /api/clans/[id]/join végpontra. A felhasználó csatlakozik a klánhoz.

- leaveClan(id, token): POST kérés az /api/clans/[id]/leave végpontra. A felhasználó kilép a klánból.

- kickMember(clanId, memberUsername, token): POST kérés az /api/clans/[clanId]/kick/[memberUserName] végpontra. Egy tagot kizárnak a klánból (csak a klántulajdonos).

- getFamousClans(): GET kérés az /api/famous-clans végpontra. A népszerű klánok listáját adja vissza.

9.4.3 MessageService – Üzenetkezelési szolgáltatás:

A MessageService az üzenetcsatorna kezelésére szolgál:

- getMessage(token, clanId): GET kérés az /api/messages/[clanId] végpontra. Az adott klán összes üzenetét lekéri az időbélyeggel és feladó információval.

- createMessage(token, clanId, message): POST kérés az /api/messages/[clanId] végpontra. Új üzenetet küld az adott klánba. Az üzenet szövegének "message" key alatt van átadva az objektumban.

9.4.4 GameService – Játék kezelési szolgáltatás:

A GameService a játékok kezelésére szolgál és BehaviorSubject-et használ az állapotkezeléshez:

- getGames(search): GET kérés az /api/games végpontra. Opcionálisan keresési paraméterrel szűrhet játékokat név alapján.

- uploadGame(gameData): POST kérés az /api/admin/upload-game végpontra. Új játékot tölt fel (adminok számára).

- getFamousGames(): GET kérés az /api/famous-games végpontra. A népszerű játékokat adja vissza logójukkal és klánok számával.

- updateSelectedGames(gameIds): Frissíti a selectedGameIds BehaviorSubject-et. Ez az igény a Navbar komponensből érkezik, amikor a felhasználó szűrni szeretne.

A GameService egy Observable-t tesz közzé (selectedGameIds$), amelyre más komponensek feliratkozhatnak az aktuális kiválasztott játék szűrések megtudásához.

9.4.5 AdminService – Adminisztratív funkciók kezelése:

Az AdminService az adminisztratív műveleteket kezeli:

- getIsAdmin(token): GET kérés az /api/admin/is-admin végpontra. Ellenőrzi, hogy az adott token admin jogosultságokkal rendelkezik-e.

- createGame(gameData, token): POST kérés az /api/admin/upload-game végpontra. Új játékot tölt fel FormData-val, amely a játék nevét és logót tartalmazza.

9.5 Felhasználófelületi komponensek részletesen

9.5.1 App komponens – Az alkalmazás gyökérkomponense:

Az App komponens az alkalmazás legfelső szintű komponense, amely tartalmazza a teljes alkalmazás struktúráját. Az alábbi elemeket rendereli:

- Navbar komponens: Az alkalmazás felső navigációs sávja

- RouterOutlet: Az aktuális útvonalnak megfelelő komponens jelenik meg

- Footer komponens: Az alkalmazás lábléce

Az App komponens függőséggel rendelkezik a UserService-hez az autentikációs modal megjelenítésének kezeléséhez.

9.5.2 Főoldal komponens – A kezdőoldal:

A főoldal komponens az alkalmazás kezdőoldalát jeleníti meg. Az ngOnInit-ben meghívja a fetchFamousGames() és fetchFamousClans() metódusokat. Ezek a GameService és ClanService-ből lekérik az adatokat. A komponens ezután dinamikusan hozzáadja a játék-elemeket és klán-elemeket a DOM-hoz az innerHTML segítségével, amely osztályok és strukturája alapján CSS-sel stílusozható.

A komponens megjelenít:

- 5 népszerű játékot a logóval, játék nevével és klánok számával

- 5 népszerű klánt a klánnevével, társított játék logójával és tagok számával (ezekre rá kell vinni a kurzort és felugranak a fentiekben megemlített dolgok)

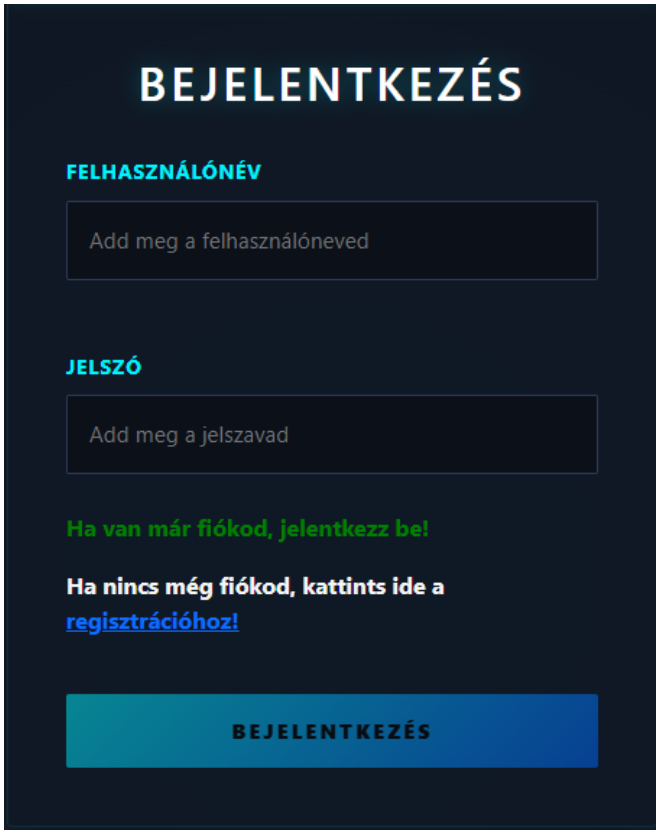
Az adatok a backend-ből érkeznek, amely a database-ből származnak.

9.5.3 Bejelentkezés komponens – Autentikációs felület:

A login komponens a bejelentkezési felületet biztosítja. Az alábbi funkcionalitásokkal rendelkezik:

- Felhasználónév beviteli mező (ngModel segítségével kötött)
- Jelszó beviteli mező (password type, ngModel-lel kötött)
- Bejelentkezési gomb, amely az onSubmit() metódust hívja meg
- Hibamegjelenítés, amely az errorMessage signal-ből származik
- Link a regisztrációs oldalra azoknak, akiknek nincs fiókjuk
- Automatikus átirányítás a főoldal-ra, ha a felhasználó bejelentkezett (ngOnInit-ben)

Az onSubmit metódus a UserService.login() metódust hívja meg a felhasználónév és jelszó párral. Ha sikeres, a token localStorage-be kerül, majd a felhasználó átirányításra kerül a főoldalra. Hiba esetén az errorMessage megjelenítésre kerül.

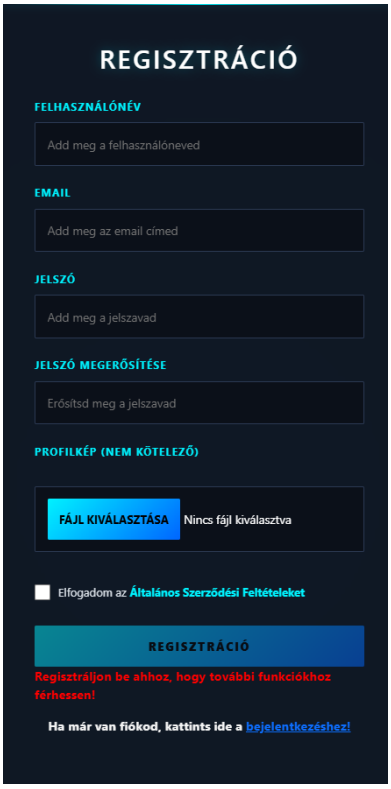


9.5.4 Regisztráció komponens – Regisztrációs felület:

A regisztráció komponens a felhasználó regisztrációs felületét jeleníti meg:

- Felhasználónév beviteli mező
- Email cím beviteli mező
- Jelszó beviteli mező
- Jelszó megerősítésének mezője
- Profilkép feltöltési input (file type)
- Feltételek elfogadásnak checkboxja
- Regisztrációs gomb

Az onSubmit() metódus először ellenőrzi, hogy a jelszó és megerősítésük megegyezik-e. Ezután FormData-t hoz létre, amely tartalmazza az összes felhasználói adatot és a feltöltött képfájlt. A UserService.register() metódust hívja meg ezzel a FormData-val. Sikeres regisztráció esetén az oldal átirányul a login oldal-ra.

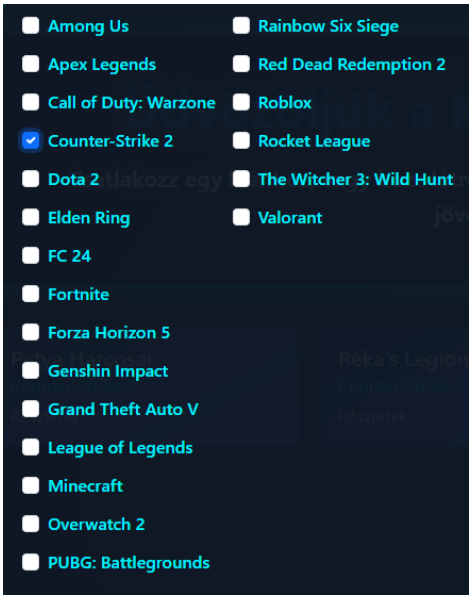


9.5.5 Klánok komponens – Klán böngészési és kezelési oldal:

A klánok komponens az alkalmazás legalapvetőbb komponensei közül az egyik legfontosabb. Az alábbi funkcionalitásokkal rendelkezik:

- Az összes klán listázása

- Klánok szűrése kiválasztott játékok alapján (a Navbar-ből kapott szűrési adatok alapján)



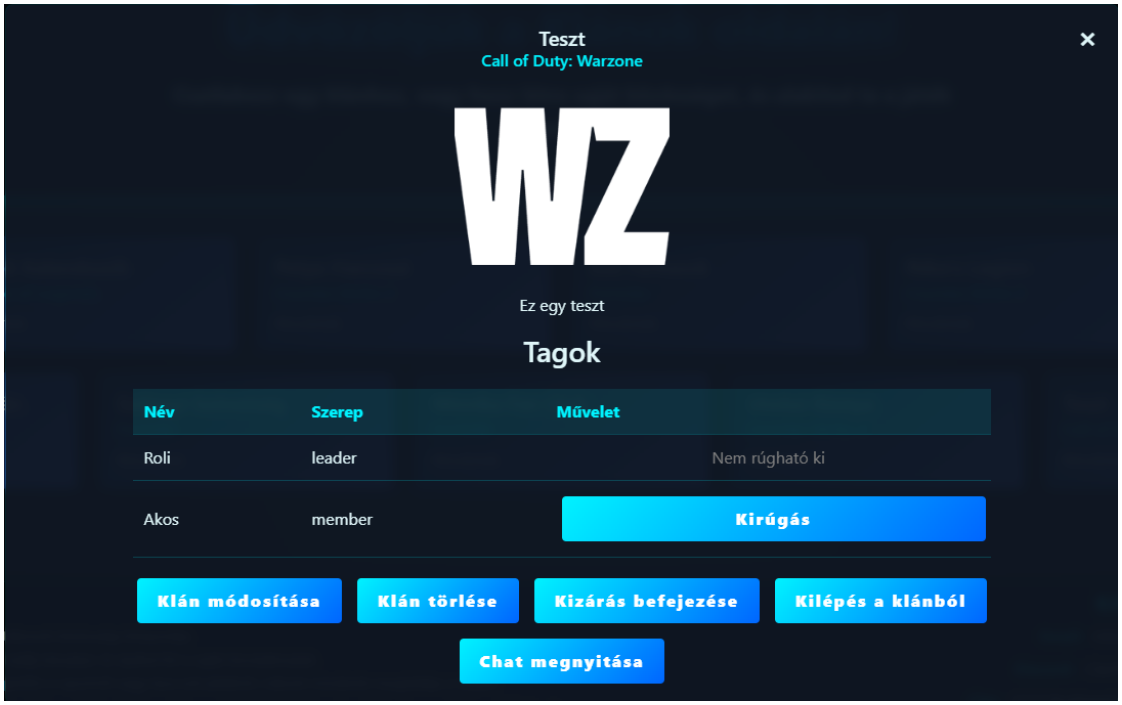
- Klán részletek modal megtekintés, amely megjelenít:

- Klán nevét és leírást

- Klán tagok listáját a1 szerepükkel

- Csatlakozási, kilépési, szerkesztési és törlési jogosultságokat (szerkesztési jogosultság alapján)

- Felhasználók kizárása klánokból (csak szerkesztési jogosultságokkal rendelkezőknek)

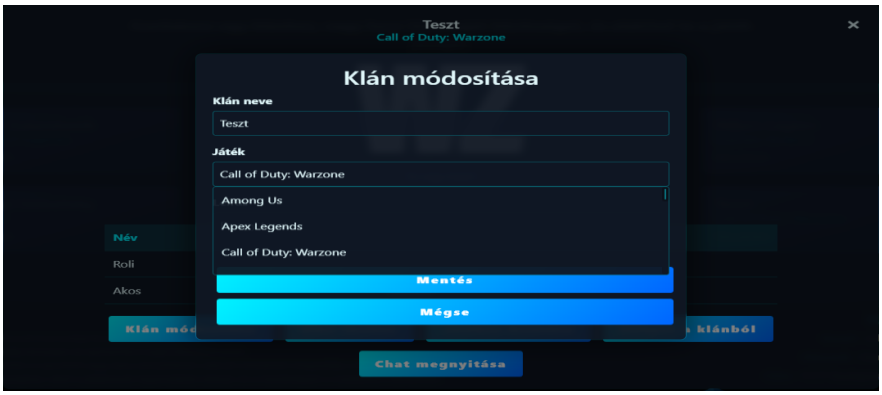


- Üzenetcsatorna megnyitása

- Dinamikus játéknév-keresési automatikus kitöltés szerkesztéskor

A komponens az ngOnInit-ben feliratkozik a GameService.selectedGameIds$ Observable-re, hogy megtudja, mely játékok alapján szűrjön. Az applyFilter() metódus az allClans tömböt szűri a selectedGameIds alapján.

Az edit modalt az openEditModal() metódus nyitja meg, amely lehetővé teszi a klán nevének, leírásának és játékának módosítását. A szerkesztés az updateClan() metódus meghívásával történik.



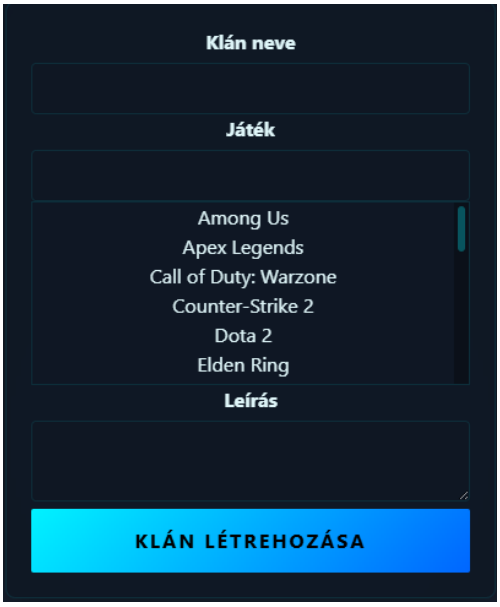
9.5.6 Klán létrehozás komponens- Új klán létrehozás:

A klán létrehozás komponens az új klánok létrehozásának felületét biztosítja:

- Klán neve beviteli mező

- Játék választó autokiegészítéssel (dinamikus keresés a GameService segítségével)

- Klán leírás szövegterület



Az ngOnInit-ben a komponens ellenőrzi, hogy a felhasználó bejelentkezve van-e (localStorage token). Ha nincs, megjeleníti az autentikációs modalt. Ezután a fetchGames() metódus lekéri az összes játékot.

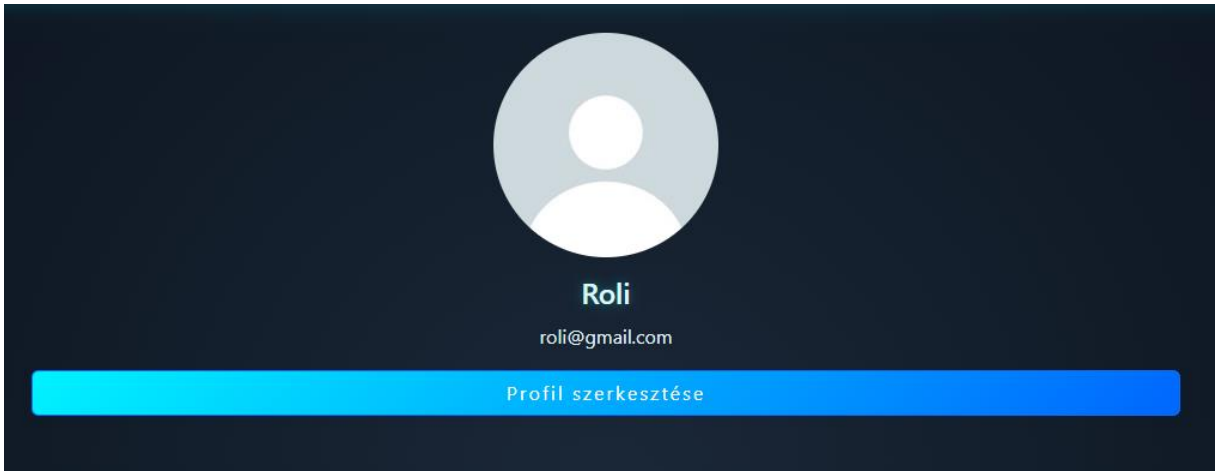
Az onGameInput() metódus valósítja meg az autokiegészítést. Amikor a felhasználó beír egy játéknevet, a GameService.getGames(value) metódust hívja meg, amely szűrt listát ad vissza.

Az onSubmit() metódus validálja, hogy mindkét kötelező mező kitöltve van-e, majd a ClanService.createClan() metódust hívja meg. Ha sikeres, az oldal átirányul a /clans oldal-ra.

9.5.7 Profil komponens – Felhasználói profil szerkesztés:

A Profil komponens a felhasználó személyes profil adatainak megtekintésének és szerkesztésének felületét jeleníti meg:

- Felhasználónév megjelenítése
- Email cím megjelenítése
- Profilkép megjelenítése
- Szerkesztés mód aktiválása, amely lehetővé teszi:



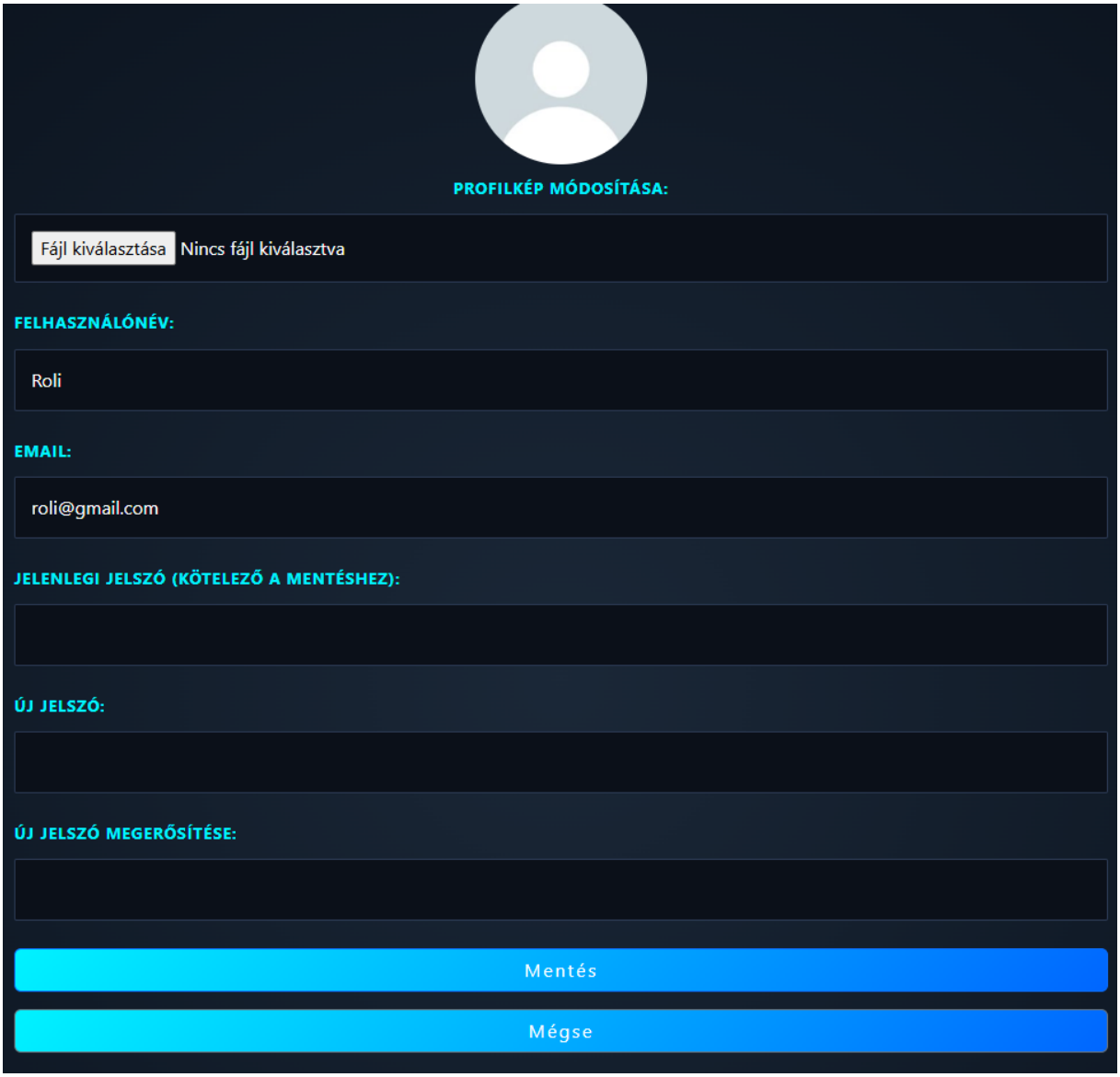
- Felhasználónév módosítást

- Email cím módosítást

- Profilkép feltöltést

- Jelenlegi jelszó megadást (biztonsági aspektus)

- Új jelszó beállítást és megerősítést

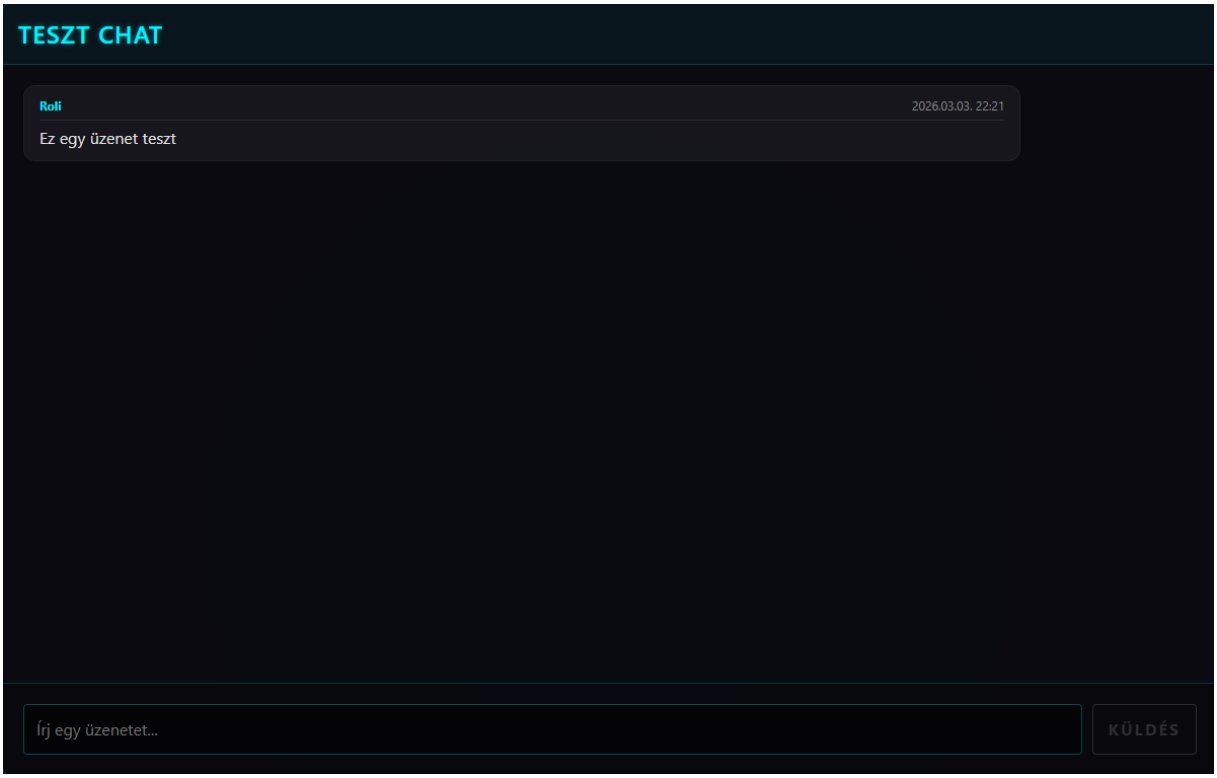


Az ngOnInit-ben a komponens meghívja a UserService.getProfile() metódust az aktuális felhasználó adatainak lekérésére. Az updateProfile() metódus FormData-t hoz létre és a UserService.updateProfile() metódust hívja meg.

9.5.8 Message komponens – Klánspecifikus üzenetcsatorna:

A Message komponens egy klánspecifikus üzenetcsatornát jeleníti meg az üzenetek listázásának és új üzenetküldésnek lehetőségével:

- Az összes üzenet megjelenítése szöveg, feladó neve és időbélyeggel
- Üzenetbeviteli mező és "Küldés" gomb
- Automatikus görgetés az utolsó üzenethez
- Klán neve megjelenítése



Az ActivatedRoute-ot használva a komponens lekéri a clanId paramétert az URL-ből. Az ngOnInit-ben meghívja a loadMessages() metódust, amely a MessageService.getMessage() metódust hívja meg. Az AfterViewChecked hook segítségével az üzenetlista automatikusan görgetődik az utolsó üzenetre.

A sendMessage() metódus validálja az üzenet szövegét, majd a MessageService.createMessage() metódust hívja meg. Sikeres küldés után az üzenetek listája frissül és a beviteli mező törlődik.

9.5.9 Admin komponens - Adminisztratív panel:

Az Admin komponens az adminisztratív panel, amely új játékok feltöltésére szolgál:

- Admin jogosultság ellenőrzése az isAdmin() metódussal az ngOnInit-ben
- Ha nincs admin jogosultság, átirányít az alkalmazás kezdőoldalára
- Játék neve beviteli mező
- Játék logó feltöltési input (nem kötelező, de ha nincs kép feltöltve, akkor a gameNotFound.jpg lesz az alapértelmezett.)
- Feltöltés gomb

Az uploadGame() metódus FormData-t hoz létre, amely a játék nevét és logót tartalmazza, majd az AdminService.createGame() metódust hívja meg. Az error handling az alábbi HTTP státuszkódokat kezel: 403 (nincs jogosultság), 409 (már létezik a játék), 400 (hiányzó adatok).

9.5.10 Ászf komponens – Általános Szerződési Feltételek:

Az aszf komponens egy statikus oldal az Általános Szerződési Feltételek megjelenítésére. A komponens az aszf.html sablonban van definiálva, amely a szerződés szövegét tartalmazza.

9.5.11 Navbar komponens – Navigációs sáv:

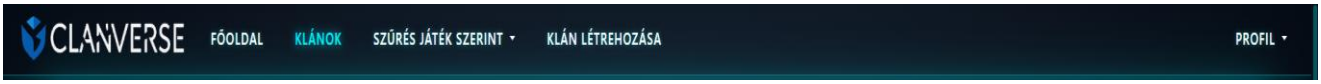
A Navbar komponens az alkalmazás felső navigációs sávja, amely az alábbi funkcionalitásokkal rendelkezik:

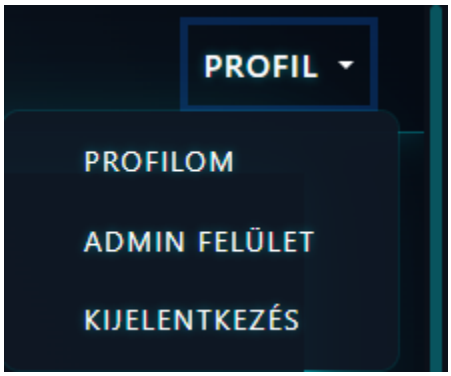
- Logo megjelenítése és link a kezdőoldalra
- Bejelentkezési állapot jelzése
- Admin panelhez való link (csak admin felhasználónak)
- Profilhoz való link (bejelentkezve lévőknek)

- Kijelentkezési gomb (bejelentkezve lévőknek)

- Klánok oldal navigáció

- Játékszűrő (csak a /clans oldalon jelenik meg)





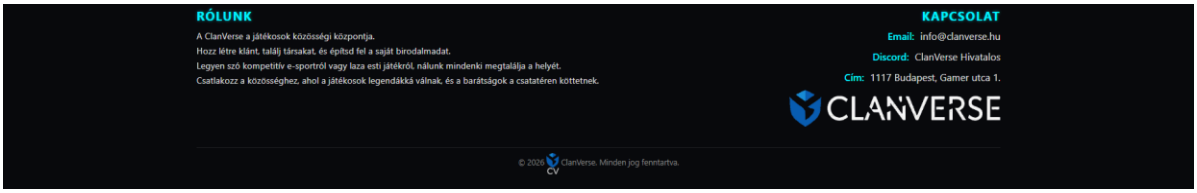
A komponens az ngOnInit-ben megvizsgálja a localStorage-ben tárolt token-t. Ha létezik, a felhasználó bejelentkezve van. Az AdminService.getIsAdmin() metódust hívja meg az admin státusz ellenőrzéséhez.

A RouterEvents-re feliratkozva a komponens nyomon követi az aktuális URL-t. Ha a felhasználó a /clans oldalra navigál, az fetchGamesForFilter() metódus meghívásra kerül az összes játék lekérésére, hogy az felhasználó szűrhessen közöttük.

Az onFilterChange() metódus a GameService.updateSelectedGames() metódust hívja meg az aktuálisan kiválasztott játékok listájával. Ez az információ a Clans komponensre kerül a szűréshez.

9.5.12 Footer komponens – Alkalmazás lábléce:

A Footer komponens az alkalmazás alsó lábléce, amely szerzői jogi információkat, a „rólunk” információját és kapcsolattartási adatokat, utolsósorban logókat tartalmaz. Ez egy statikus komponens, amely az footer.html sablonban van definiálva.



9.6 Alkalmazás konfigurációi és inicializálás

9.6.1 App config – Az alkalmazás konfigurációs fájlja:

Az app.config.ts az alkalmazás indítási konfigurációját definiálja:

- provideZoneChangeDetection: Az Angular Zone.js beállítása az alkalmazás-szintű változásdetektáláshoz. Az eventCoalescing: true beállítás csökkenti a felesleges kiértékeléseket.

- provideRouter: Az alkalmazás útvonalazásának regisztrálása

- provideHttpClient: A HTTP kliens regisztrálása az authInterceptor-ral

- APP_INITIALIZER: Egy függvény, amely az alkalmazás indulásán fut le. Ez az initializeApp függvény a UserService-et használva validálja a tárolt token-t, ha létezik.

9.7 Styling és CSS keretrendszer

Az alkalmazás a Bootstrap 5 CSS keretrendszert használja az alapvető reszponzív dizájnhoz. A Bootstrap CSS az angular.json-ben van konfigurálva az összes komponensre alkalmazható globális stílusként. Az alkalmazás és komponensek saját CSS fájlokkal rendelkeznek az alkalmazás-specifikus stílusokhoz:

- src/styles.css: Az alkalmazás globális stílusai

- src/app/app.css: Az App komponens stílusa

- src/app/component/[komponensnév]/[komponensnév].css: Az egyes komponensek specifikus stílusa

Az egyes komponensek kapnak encapsulation: ViewEncapsulation.None beállítást, amely lehetővé teszi, hogy a komponens CSS-je az egész alkalmazásra vonatkozzon.

9.8 HTTP kommunikáció és API integrációs minta

9.8.1 API végpontok és útvonalak:

Az alkalmazás a következő API végpontokkal kommunikál a localhost:3000/api-n futó backend-del:

/users: Felhasználó kezelés (register, login, logout, validate, profile, update profile)

/clans: Klán kezelés (create, get, get by id, update, delete, join, leave, kick)

/messages: Üzenetkezelés (get messages, create message)

/games: Játék kezelés (get games, search games, get famous games)

/admin: Adminisztratív funkciók (is admin, upload game)

9.8.2 Error handling és hibakezelés:

Az alkalmazás az RxJS subscribe metódus error callback-jét használja a hibakezelésre. A komponensek általában alert-kel vagy megjelenített hibaüzenetekkel reagálnak a hibákra. Az authInterceptor a 401-es hibákat (Unauthorized) speciális módon kezeli, eltávolítva a token-t és megjelen az autentikációs modalt.

9.9 Forma kezelés és validáció

Az alkalmazás az Angular FormsModule-jét használja az egyszerű kétirányú adatkötéshez (ngModel). Bár a ReactiveFormsModule az összetettebb formalogikához ajánlott lenne, az alkalmazás az egyszerű FormsModule-ra támaszkodik.

Az alapvető validáció az UI-ban történik, például:

- Jelszónak egyeznie kell a megerősítővel a Regisztráció és Profil komponensekben
- Kötelező mezők kitöltésének ellenőrzése az onSubmit metódusokban

A fájlfeltöltések FormData objektummal történnek, amely lehetővé teszi a bináris adatok küldését a HTTP POST/PUT kérésekben. Ez szükséges a képfájlok feltöltéséhez.

9.10 Biztonsági szempontok és lehetséges sebezhetőségek

9.10.1 Token tárolás biztonsága:

A JWT token jelenleg a localStorage-ben tárolódik, amely potenciális XSS (Cross-Site Scripting) sebezhetőséget jelent. A localStorage-ben tárolt adat bármely JavaScript-code hozzáfér. Éles környezetben ajánlott a HttpOnly flag-gel rendelkező cookie-t használni, amely JavaScript-ből nem elérhető.

9.10.2 Jelszókezelés:

Az alkalmazás a jelszót a backend-nek küldi, amely felelős a jelszó hasheléséért (bcrypt vagy hasonló algoritmus). Az alkalmazás nem végez jelszókezelést, csak továbbítja a hashelt jelszót.

9.10.3 Content Security Policy (CSP):

Egy jól definiált CSP fejléc a backend-en segít megvédeni az alkalmazást az XSS és más szöveges támadásokkal szemben.

9.11 Teljesítmény optimalizálási módszerek

9.11.1 Lazy Loading komponensek:

Az alkalmazás az Angular Router lazy loading funkciójával él. Ez azt jelenti, hogy az egyes komponensek csak akkor kerülnek letöltésre a böngészőbe, amikor az adott útvonal aktiválódik. Ez jelentősen csökkenti az alkalmazás indítási méretét és feldolgozási idejét.

9.12 Összefoglaló

A ClanVerse frontend egy modern, jól strukturált Angular alkalmazás, amely lehetővé teszi a felhasználók számára klánok létrehozását, csatlakozását és kommunikációját valós idejű üzenetkezelés révén. Az alkalmazás moduláris architektúrára épül, amely lehetővé teszi az egyszerű karbantartást, tesztelést és kiterjesztést. A komponensek tiszta szeparációt tartanak a sablonok, logika és stílusok között. Az alkalmazás Bootstrap alapú reszponzív dizájnt használ, amely jó felhasználói élményt biztosít az összes eszközön. Az autentikációs rendszer biztonságos JWT token alapokra épül, bár éles környezetben további biztonsági intézkedések ajánlottak. Az alkalmazás jó alapot biztosít a jövőbeli fejlesztésekre és skálázásra.

X. Projekt szerkezete

A projekt architektúrája logikusan szervezett könyvtárstruktúrába rendeződik, biztosítva a könnyű navigációt a forráskódban.

ClanVerseProjekt_BodiszSedlakSimko/

├── backend/ (Az Express/Node.js API réteg)

│ ├── controllers/ (Üzleti logika, adatok kinyerése és manipulációja)

│ ├── models/ (Sequelize tábladefiníciók, relációk, ORM)

│ ├── routes/ (Végpontok definiálása, middleware-ek láncolása)

│ ├── middleware/ (HTTP kéréseket elfogó szűrők: auth, képfeltöltők)

│ ├── data/ (Nyers JSON adatok a teszteléshez)

│ ├── database/ (XAMPP adatbázis összekapcsolása)

│ ├── seeds/ (Az adatbázist induláskor automatikusan feltöltő szkriptek)

│ ├── utils/ (Globális segédmodulok: config, jwt generátor, HTTP státuszkód kezelő)

│ ├── uploads/ (A szerver által fogadott és statikusan kiszolgált fájlok)

│ ├── server.js (A backend belépési pontja és főszervezője)

│ ├── .env (környezeti változók)

│ └── package.json (Node.js csomagfüggőségek)

├── frontend/ (Az Angular alapú SPA Kliens)

│ ├── src/

│ │ ├── app/

│ │ │ ├── component/ (A UI építőkockái: navbar, profil, chatablak, login űrlap, stb.)

│ │ │ ├── services/ (Az API-al kommunikáló adatinjektorok és HTTP kliensek)

│ │ │ ├── guards/ (Kliensoldali végpontvédelem: navigáció blokkolása bejelentkezés nélkül)

│ │ │ ├──interceptors/ (A HTTP kérésekbe beavatkozó logika, minden kéréshez hozzáfűzi a JWT-t)

│ │ │ └── app.routes.ts (Az oldalak közötti navigáció térképe)

│ │ ├── assets/ (Projekt logója és bannerje)

│ │ └── styles.css (Globális CSS szabályok és keretrendszer felülírások)

│ └── angular.json (Az Angular CLI motor konfigurációs fájlja)

XI. Munkamenet és csapatfelosztás

A szoftver fejlesztése az Agilis szoftverfejlesztési módszertan alapelveire épült. Emellett a „Shift left” módszert alkalmaztuk az elkészítés során.

Munkamegosztás:

- Sedlák Ákos: Backend architektúra, relációs adatbázis tervezése (ERD Diagram), API végpontok (Express + Sequelize) teljes körű fejlesztése, adatvédelmi algoritmusok, projektmenedzsment feladatok.
- Simkó Roland: Kliensoldal fejlesztése: „Clans”, klán létrehozás, admin, bejelentkezés és regisztrációs oldal elkészítése. Színpaletta elkészítése és beépítése.
- Bódisz Loránd: Kliensoldal fejlesztése: navigációs sáv és lábrész elkészítése (Bootstrap), főoldal, chat és profil komponens elkészítése.

A projekt tervezése során úgy döntöttünk, hogy a csapat egy backend és adatbázis fejlesztőből, valamint két frontend fejlesztőből álljon. Ennek oka a feladatok optimális elosztása volt: a backend fejlesztő látja át a legpontosabban az adatbázis szerkezetét és a tárolandó értékeket, a kliensoldal (felhasználói felület) megtervezése és leprogramozása viszont arányaiban sokkal több időt és erőforrást igényel. Ez a felállás biztosította a leghatékonyabb munkavégzést.

XII. Felhasználói esetek és követelmények (User Stories)

A kódolás megkezdése előtt meghatároztuk a rendszer funkcionális követelményeit. A funkciókat "User Story" formátumban fogalmaztuk meg, fókuszban tartva a végfelhasználói igényeket.

- Vendég (Látogató): Szeretném látni a főoldalon a statisztikákat (legnépszerűbb játékok), hogy eldönthessem, érdemes-e regisztrálnom.
- Normál Felhasználó: Szeretnék biztonságosan bejelentkezni, és bármikor frissíteni a profilképemet. Szeretném átlátni a klánok listáját, keresni köztük, és egy kattintással csatlakozni egyhez. Szeretnék üzeneteket váltani a klántársaimmal a saját felületünkön.
- Klánvezető (Leader): Szeretném a saját klánomat testreszabni, leírást adni neki. Szeretném a tagságomat menedzselni, és kizárni azokat, akik megsértik a szabályzatot. Szeretném a klánt véglegesen feloszlatni, ha szükséges.
- Rendszergazda (Admin): Szeretnék egy dedikált, rejtett felületet, ahonnan a teljes platform játékkatalógusát tudom bővíteni új játékok hivatalos képének feltöltésével.

XIII. Szoftvertesztelés jegyzőkönyv

A szoftver minőségbiztosítása során több rétegű tesztelési módszertant alkalmaztunk a "bugok" kiszűrésére.

- Egységtesztek (Unit) és Biztonsági tesztek: Ellenőriztük a jelszó-titkosító algoritmust. Leteszteltük, hogy lejárt, vagy módosított JWT token esetén a middleware ("auth.js") azonnal megszakítja-e az adatfolyamot egy 401 Unauthorized hibával.
- API és Integrációs Tesztelés (Postman és összekötés után): Az összes végpontot extrém adatbevitellel teszteltük. Például megpróbáltunk képként feltölteni egy ".txt" fájlt a profilkép módosításánál, amit a szerver (Multer) sikeresen blokkolt 400 Bad Request hibával. Vizsgáltuk az SQL Injection lehetőségeket a keresőmezőkben, de a Sequelize ORM minden ilyen kísérletet ártalmatlanított.
- A frontend E2E (End-to-end) és manuális tesztelése: A frontend Angular Guard-jának tesztelése során megkíséreltünk bejelentkezés nélkül egy védett URL-re (pl. "/profile") navigálni, de az alkalmazás hibátlanul megjelenítette a felugró ablakot (modal-t), ami csak a bejelentkezés oldalra tud navigálni. Teszteltük a reszponzivitást mobilnézetben, biztosítva, hogy a Bootstrap rácsrendszer helyesen tördeli a klánok kártyáit (Cards).
- Terheléses tesztelés: Iskolában elindítottuk a projektet minden porton és az osztályból többen felcsatlakoztak és használták a webalkalmazást. Töltöttek fel képeket, regisztráltak, létrehoztak klánokat és chat-eltek.
- Dokumentáció és alkalmazás tesztje külsős által: A projekt telepítésre, futtatására és kipróbálására megkértünk olyan ismerősöket, akik nem értenek az informatikához.

XIV. Kódanalízis (backend és adatbázis)

Ebben a fejezetben fejlesztői szempontból elemezzük a forráskódot. Bemutatjuk a legkritikusabb algoritmusokat és tervezési mintákat, amelyek az alkalmazás magját adják.

14.1. Adatbázis-kapcsolat és szinkronizáció

A rendszer adatbázisát egy helyi XAMPP szerveren hoztuk létre (MariaDB). A Node.js backend ehhez a relációs adatbázishoz a Sequelize objektum-relációs leképzőn keresztül kapcsolódik, amely automatizálja a lekérdezéseket és a táblák szinkronizációját.

Fájl: database/sequelize.js (Részlet)

```
1  const { Sequelize } = require('sequelize');
2  const config = require('../utils/config');
3
4  const sequelize = new Sequelize(
5      config.db.name,
6      config.db.username,
7      config.db.password,
8      {
9          host: config.db.host,
10         dialect: config.db.dialect,
11         logging: false,
12
13         timezone: '+01:00',
14
15         dialectOptions: {
16             charset: 'utf8mb4',
17             useUTC: false,
18             dateStrings: true,
19             typeCast: true,
20         },
21
22         define: {
23             timestamps: true,
24             collate: 'utf8mb4_hungarian_ci',
25         },
26     }
27 );
```

Kódanalízis: A "charset: 'utf8mb4'" és a "collate: 'utf8mb4_hungarian_ci'" paraméterek kritikusak egy nemzetközi, vagy chat funkcióval rendelkező rendszernél. Ez teszi lehetővé, hogy az adatbázis hiba nélkül dolgozza fel a speciális és ékezetes karaktereket, valamint az emojikat. A "timezone" beállítása garantálja, hogy a "timestamps: true" által automatikusan generált "createdAt" időbélyegek mindig a magyar időzónát tükrözzék.

14.2. Dinamikus Adatmodellek és Táblakapcsolatok

A táblák sémája osztályként van reprezentálva a memóriában. Vegyük szemügyre a legfontosabb kapcsolótáblát, amely a felhasználók és a klánok viszonyát (Több-a-többhöz) oldja fel.

Fájl: models/clanMembers.js (Részlet)

```
1  const { DataTypes } = require('sequelize');
2  const sequelize = require('../database/sequelize');
3
4  const ClanMembers = sequelize.define('ClanMembers', {
5      id: {
6          type: DataTypes.INTEGER(11),
7          autoIncrement: true,
8          primaryKey: true
9      },
10     clanId: {
11         type: DataTypes.INTEGER(11),
12         allowNull: false,
13         references: {
14             model: 'clans',
15             key: 'id'
16         },
17         onDelete: 'CASCADE',
18         onUpdate: 'CASCADE'
19     },
20     userId: {
21         type: DataTypes.INTEGER(11),
22         allowNull: false,
23         references: {
24             model: 'users',
25             key: 'id'
26         },
27         onDelete: 'CASCADE',
28         onUpdate: 'CASCADE'
29     },
30     role: {
31         type: DataTypes.STRING(100),
32         allowNull: false,
33         defaultValue: 'member'
34     }
```

Kódanalízis: Az "onDelete: 'CASCADE'" szabály ereje itt mutatkozik meg. Adatbázis szinten delegáljuk a felelősséget a MariaDB-re: amennyiben a kód egy másik része töröl egy klánt (vagy egy felhasználót), a MariaDB automatikusan "eltakarítja" a hozzá tartozó összes tagsági rekordot ebből a táblából. Ez tehermentesíti a Node.js szervert, és garantálja a 100%-os relációs adatintegritást.

14.3. Hitelesítés, Jogosultság és Végpontvédelem (Auth Middleware)

Minden védett végpont előtt a rendszer lefolytat egy ellenőrzést, amely az állapotmentes (stateless) JWT tokent vizsgálja.

Fájl: middleware/auth.js (Részlet)

```
1  const jwt = require('jsonwebtoken');
2  const config = require('../utils/config');
3
4  const { Code401 } = require('../utils/statusCode');
5
6  module.exports = (req, res, next) => {
7      const authHeader = req.headers['authorization'] || req.headers['Authorization'];
8      if (!authHeader) {
9          return Code401(null, req, res, next, "Nincs jogosultsága a felhasználónak (nincs token).");
10     }
11
12     const parts = authHeader.split(' ');
13     if(parts.length !== 2 || parts[0] !== 'Bearer'){
14         return Code401(null, req, res, next, "Nincs jogosultsága a felhasználónak (hibás token).");
15     }
16
17     try{
18         const payload = jwt.verify(parts[1], config.secretKey);
19         req.user = payload;
20         next();
21     }catch(err){
22         if(err.name === 'TokenExpiredError'){
23             return Code401(null, req, res, next, "Nincs jogosultsága a felhasználónak (lejárt token).");
24         }
25         return Code401(err, req, res, next, "Nincs jogosultsága a felhasználónak (érvénytelen token).");
26     }
27 };
```

Kódanalízis: A függvény kiolvassa a HTTP fejlécből a tokent, szétbontja a szabványos "Bearer [token]" struktúrát, majd a szerver titkos "secretKey"-jével megkísérli a dekódolást (jwt.verify). Ez egy szinkron művelet, amely matematikai pontossággal megmondja, hogy a token hamisított-e. Sikeres validáció esetén a "payload" (ami tartalmazza a felhasználó ID-ját, nevét és azt, hogy admin-e) bekerül a "req.user" változóba, amelyet a Controller a következő fázisban már szabadon felhasználhat az adatbázis lekérdezésekhez.

14.4. Szigorúan Ellenőrzött Fájlfeltöltés (Multer)

A képek szerverre juttatása a legsebezhetőbb pontok egyike. Ezt a Multer modul finomhangolásával védtük ki.

Fájl: middleware/uploadPfp.js (Részlet)

```
1  const storage = multer.diskStorage({
2      destination: (req, file, cb) => {
3          cb(null, 'uploads/profilePictures/');
4      },
5      filename: (req, file, cb) => {
6          cb(null, Date.now() + path.extname(file.originalname));
7      }
8  });
9  function checkFileType(file, cb) {
10     const filetypes = /jpeg|jpg|png/;
11     const extname = filetypes.test(path.extname(file.originalname).toLocaleLowerCase());
12     const mimetype = filetypes.test(file.mimetype);
13
14     if(mimetype && extname){
15         return cb(null, true);
16     } else {
17         return cb(new Error('Csak kép fájlokat lehet feltölteni!'));
18     }
19 }
20
21 const uploadPfp = multer({
22     storage: storage,
23     limits: { fileSize: 5 * 1024 * 1024 }, // 5MB limit
24     fileFilter: (req, file, cb) => { checkFileType(file, cb); },
25 });
```

Kódanalízis: Három védvonalat építettünk be.

1: A "limits" megakadályozza a memória túlcsordulását (Max 5MB).

2: A "checkFileType" kettős Regex vizsgálatot tart: ellenőrzi a név kiterjesztését ÉS a hálózaton utazó bináris MIME-típust. Így a trükkösen átnevezett "vírus.jpg.php" fájlok is elbuknak.

3: A fájl új, generált Unix időbélyeg (Date.now) nevet kap, eltüntetve az esetlegesen kártékony eredeti fájlneveket.

14.5. Komplex Üzleti Logika: Fail-Fast (Gyors Bukás)

Minta a Controllerekben: A controllerek feladata a logikai peremfeltételek ellenőrzése. Nézzük a tagok kizárásáért felelős algoritmust.

Fájl: controllers/clans.controller.js (Részlet)

```
exports.kickMember = async (req, res, next) => {
    try {
        const { userId } = req.user;
        const { id, memberUserName } = req.params;

        const clan = await Clans.findByPk(id);
        if(!clan) return Code404("Nem található ilyen klán", req, res, next);

        const isLeader = await ClanMembers.findOne({ where: { clanId: id, userId: userId, role: 'leader'}});
        if(!isLeader) return Code403("Nincs jogosultságod a tagok kirúgásához", req, res, next);

        const user = await Users.findOne({ where: { username: memberUserName } });
        if(!user) return Code404("Nem található a felhasználó", req, res, next);

        const member = await ClanMembers.findOne({ where: { clanId: id, userId: user.id}});
        if(!member) return Code404("Az adott felhasználó nem tagja a klánnak", req, res, next);

        if(member.role === 'leader') return Code403("Nem tudod kirúgni a klán vezetőjét!", req, res, next);

        await member.destroy();
        return res.status(200).json({ message: "Sikeresen kizártad a tagot a klánból" });
    } catch (err) {
        return Code500(err, req, res, next);
    }
}
```

Kódanalízis: Ez a függvény az ipari "Fail Fast" tervezési mintát implementálja. Öt egymást követő logikai sorompót (if statement) tartalmaz. Nem pazarol szervererőforrást felesleges lépésekre: amint az első feltétel megbukik (pl. nincs ilyen klán, vagy nincs 'leader' jog), a végrehajtás azonnal megszakad (early return), és a szabványosított hibaüzenet-generátor (Code403, Code404) visszaküldi a hibát a kliensnek. Csak a minden szempontból helyes kérések jutnak el az adatbázis "destroy()" utasításáig.

14.6. A Központosított Hibakezelés Elve

A projektben elkerültük, hogy a több tucat controller fájlban mindenhol külön újraírjuk a res.status(X).json(...) struktúrát. Ehelyett egy magasabb rendű függvényt készítettünk. Ezzel egyébként jól hasznosítjuk az OOP elveket is.

Fájl: utils/statusCode.js

```
1  const handleError = (status, uzenet) => {
2      return (error, req, res, next, reason) => {
3          if(error) console.error(`Hiba (${status}): ${error}`);
4          if(!reason || reason.length < 2){
5              return res.status(status).json({ status, message: uzenet });
6          }
7          return res.status(status).json({ status, message: uzenet, reason: reason });
8      };
9  };
10
11 const Code400 = handleError(400, "Hibás kérés.");
12 const Code401 = handleError(401, "Nincs jogosultsága a felhasználónak");
13 const Code500 = handleError(500, "Adatbázis / szerver hiba");
14 module.exports = { Code400, Code401, Code500 }; // (Rövidített változat)
15
```

Kódanalízis: A „tiszta kód” elvet követve a handleError egyszer definiálja a válaszformátumot. Ebből generáljuk ki a specifikus Code400, Code500 konstansokat. Ez garantálja, hogy az Angular frontend biztos lehet benne, hogy a hibaüzenetek szerkezete az egész rendszerben, minden végpontnál egyforma lesz, így könnyen feldolgozható.

14.7. Fő Szerver Konfiguráció és Biztonsági Házirend (CORS)

A szerver belépési pontja fogja össze az építőelemeket, és itt állítjuk be a hálózati kommunikáció legfontosabb paramétereit.

Fájl: server.js (Részletek)

```
1  const express = require('express');
2  const cors = require('cors');
3  const sequelize = require('./database/sequelize');
4  const clansRouter = require('./routes/clans.routes');
5  const app = express();
6  app.use(express.json());
7
8  app.use(
9      cors({
10         origin: 'http://localhost:4200',
11         methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
12         allowedHeaders: ['Content-Type', 'Authorization'],
13         credentials: true
14     })
15 );
16
17 app.use('/uploads', express.static('uploads'));
18 app.use('/api/clans', clansRouter);
```

Kódanalízis: Az "express.json()" teszi lehetővé a Payload adatcsomagok értelmezését. A "cors()" middleware beállítása egy nagyon erős védelmi vonal. A böngészők beépített mechanizmusa alapból blokkolná a localhost:4200 (Angular) és a localhost:3000 (Express) közötti kommunikációt. Ezzel a beállítással mi mondjuk meg a böngészőnek: KIZÁRÓLAG az Angular kliensünkről engedélyezz adatmozgást, csökkentve az oldalakon átívelő lekérdezés-hamisítások kockázatát. Az "express.static" egyetlen sorral megoldja a szerver háttértárán lévő profilképek és logók URL alapú publikus kiszolgálását.

14.8. Adatbázis Inicializálás és Biztonságos Seeding

Egy üres rendszer bemutathatatlan, ezért a fejlesztői környezet indulásakor automatizált "Seed" eljárások töltenek be előre definiált tesztadatokat JSON fájlokból. Fontos megjegyezni, hogy a tesztadatokat a mesterséges intelligenciával generáltuk szemléltetés céljából. Ami a data mappa JSON fájljaiban van, nem valós adatok!

Fájl: seeds/usersSeed.js (Részlet)

```
1  async function seedUsers() {
2      try{
3          const filePath = path.join(__dirname, '../data/users.json');
4          const fileContent = fs.readFileSync(filePath, 'utf8');
5          const usersList = JSON.parse(fileContent);
6
7          const count = await Users.count();
8          if (count >= usersList.length) return; // Ha már létezik, kilép
9
10         for (const userInfo of usersList) {
11             await Users.findOrCreate({
12                 where: { username: userInfo.username },
13                 defaults: {
14                     username: userInfo.username,
15                     email: userInfo.email,
16                     password: await bcrypt.hash(userInfo.password, config.hashIterations),
17                     isAdmin: userInfo.isAdmin || false,
18                     profilePicture: userInfo.profilePicture
19                 }
20             });
21         }
22     } catch (err){
23         throw err;
24     }
25 }
```

Kódanalízis: A seeder intelligens: lekérdezi a tábla jelenlegi sorainak számát (count), és ha kevesebb rekord van a táblában, mint a JSON fájlban, rögtön kilép, védve a szervert az IO műveletektől. A legfontosabb mérnöki döntés a "defaults" blokkban látható. A teszteléshez használt JSON fájlokban (data/users.json) a jelszavak természetesen emberi nyelven, olvasható formátumban vannak. Azonban a Seeder sosem szúrja be a nyílt jelszót. Futásidőben, mielőtt a Sequelize rögzítené az adatot, a "bcrypt.hash" lefut az adaton. Ez garantálja, hogy a fejlesztői tesztadatbázis strukturálisan és biztonságilag 100%-ban megegyezik a majdani éles környezettel. A seedelés sorrendjét (Játékok -> Felhasználók -> Klánok -> Tagok) a server.js fájlban rögzítettük a hivatkozási integritás (Foreign Key Constraints) megőrzése miatt.

XV. Továbbfejlesztési lehetőségek

A vizsgaremek első fázisa sikeresen lefektette egy skálázható szoftver alapjait. A moduláris felépítés rendkívül egyszerűvé teszi a platform jövőbeli kibővítését:

- RefreshToken beépítése: ne azzal lehessen érvényesíteni a tokent, ha a felhasználó újra bejelentkezik, hanem a refresh token segítségével a munkamenet hosszabbodjon. Létre van hozva a refreshToken módszer a kódban, de nem lett implementálva.
- A klán üzenetküldő falán egy videóhívás rendszer kiépítése, ahol tudnak képernyőt osztani, kamerát kapcsolni hanghívásban.
- Egy értesítésfül létrehozása, ahol a felhasználók értesülnek egyes eseményekről valamint, ha kizárásra kerülnek egy csapatból.
- Események szervezése klánok között (event cup): csapat a csapat elleni mérkőzések megszervezésére lehetőség.
- Az adminnak a háttérfunkciók mellett (kódban) egy grafikus frontend szerkesztés kialakítása.

XVI. Források

16.1 Bódisz Loránd források

- Angular Hivatalos Dokumentáció – komponensek, direktívák, szolgáltatások https://angular.dev/
- Angular Routing & Navigation – Guardok, lazy loading, oldal navigáció https://angular.dev/guide/routing
- Angular HTTP Client – REST kérések, Interceptorok, backend kommunikáció https://angular.dev/guide/http
- Angular Forms Guide – űrlapok kezelése, validációk https://angular.dev/guide/forms
- CSS-Tricks – gyakorlati tippek, egyedi CSS megoldások https://css-tricks.com/
- Figma Community – UI komponensek, sablonok inspirációhoz https://www.figma.com/community
- MDN Web Docs – HTML, weboldal struktúrák https://developer.mozilla.org/hu/docs/Web/HTML
- MDN Web Docs – CSS, stíluslapok, reszponzív elrendezés, animációk https://developer.mozilla.org/hu/docs/Web/CSS
- Stack Overflow – közösségi fórum programozási problémákra https://stackoverflow.com/
- The Net Ninja – frontend, Angular, RxJS videók https://www.youtube.com/c/TheNetNinja

16.2 Sedlák Ákos források

- Alapértelmezett (default) profilkép: https://hu.pinterest.com/ideas/default-profile/931256273824/ 2026.01.15 11:16
- Alapértelmezett (gameNotFound) játék kép: https://www.freepik.com/premium-vector/no-game-icon-set-forbidden-gammer-banner-vector-symbol-black-filled-outlined-style-no-games-console-sign_306725716.htm 2026.01.26 10:22
- Seeder-ezett játékok logói (minden feltöltött képet erről az oldalról töltöttem le): https://www.steamgriddb.com/ 2026.01.26 12:03
- API végpont elnevezésének konvenciói: https://restfulapi.net/resource-naming/
- Express keretrendszerű REST API felépítése, útválasztás kialakítása: https://expressjs.com/
- Sequelize ORM konfigurálása, modellek és adatbázis-kapcsolatok (XAMPP/MariaDB) beállítása: https://sequelize.org/
- Node.js futtatókörnyezet alapjai: https://nodejs.org/en
- Állapotmentes hitelesítés logikája és a tokenek szerkezete: https://www.jwt.io/
- Jelszavak védelme és titkosítása https://www.npmjs.com/package/bcrypt
- Fájlfeltöltés: https://www.npmjs.com/package/multer
- Egyes hibákra megoldás (pl. a powershell command): https://stackoverflow.com/

- Github README formázás: https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax
- JWT használata a gyakorlatban: https://youtu.be/mbsmsi7l3r4?si=Nsu4H9O1f30IiA50
- A JWT működésének elve (elméleti síkon): https://youtu.be/7Q17ubqLfaM?si=_xuY2UzbQu4eWsxk
- Mi a CORS: https://youtu.be/4KHiSt0oLJ0?si=We12YwMUtN5yheAF
- Sequelize alapok: https://youtu.be/ExTZYpyAn6s?si=tqvVCuOVFp6ZuY-v
- Sequelize és MariaDB: https://sequelize.org/docs/v7/databases/mariadb/
- Bcrypt használata: https://youtu.be/Ud5xKCYQTjM?si=xqDeDdb6w8rQizRn
- Postman dokumentáció: https://learning.postman.com/
- Postman használata: https://www.youtube.com/watch?v=MFxk5BZulVU
- CORS pontos beállítása: https://www.youtube.com/watch?v=PNtFSVU-YTI
- Dotenv: https://www.dotenv.org
- ERD Editor bővítmény dokumentációja: https://docs.erd-editor.io/docs/guide/guides/editing-start
- MariaDB dokumentáció: https://mariadb.com/docs/
- HTTP státuszkódok helyes használata: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status
- REST API tervezési minták: https://restfulapi.net/
- Hogy kommunikál az Express az Angular-ral: https://www.youtube.com/watch?v=ZyC8VveOZTY
- Nodemon fejlesztői csomag: https://www.npmjs.com/package/nodemon
- Seeder-hez használt fájl beolvasó: https://www.w3schools.com/nodejs/nodejs_filesystem.asp
- „fs” nodejs csomag: https://nodejs.org/api/fs.html

16.3 Simkó Roland források

- Angular hivatalos dokumentáció – komponensek, direktívák, keretrendszer alapjai https://angular.dev/
- Angular Routing & Navigation – oldalnavigáció, Guardok, Lazy loading https://angular.dev/guide/routing
- Angular HTTP Client – backend API-val való kommunikáció, REST kérések, Interceptorok https://angular.dev/guide/http
- Angular CLI – projekt generálása, szerver futtatása, build folyamatok https://angular.dev/tools/cli
- Fireship YouTube csatorna – rövid, áttekinthető videók modern Angular és frontend technológiákról https://www.youtube.com/c/Fireship
- Academind YouTube csatorna – Angular gyakorlati példák, komponensfejlesztés https://www.youtube.com/c/Academind

- TypeScript kézikönyv – típusos JavaScript, interface-ek, típusdefiníciók
  https://www.typescriptlang.org/docs/
- RxJS dokumentáció – aszinkron adatfolyamok, Observables, HTTP válaszok kezelése
  https://rxjs.dev/guide/overview
- Bootstrap 5.3 dokumentáció – reszponzív grid, navigációs sáv, kártyák
  https://getbootstrap.com/docs/5.3/getting-started/introduction/
- Bootstrap Icons – felhasználói felületi ikonok
  https://icons.getbootstrap.com/
- Figma – UI és UX tervezés, drótvázak, prototípusok
  https://www.figma.com/learn/design/
- Dribbble – UI inspiráció, színpaletták, modern dizájnötletek
  https://dribbble.com
- Behance – UI/UX projektek és vizuális tervek inspirációja
  https://www.behance.net/
- Canva – logó és grafikai elemek tervezése
  https://www.canva.com
- Stack Overflow – közösségi fórum programozási problémák gyors megoldására
  https://stackoverflow.com/
- Reddit – r/Angular – közösségi fórum, problémamegoldás, tippek
  https://www.reddit.com/r/angular/

- Visual Studio Code – fejlesztőkörnyezet, Angular Language Service bővítmény
  https://code.visualstudio.com/docs
- GitHub dokumentáció – verziókezelés, csapatmunka, merge konfliktusok
  https://docs.github.com/en
- Npm (Node Package Manager) – külső csomagok és függőségek kezelése
  https://docs.npmjs.com/